VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI– 590018, KARNATAKA, INDIA

A PROJECT REPORT

on

"Swaraaha: Stutter Classification and Localization System"

Submitted in partial fulfilment of the requirements for the award of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted By

Name	USN
K SHREEKRISHNA UPADHYAYA	4VP23AI020
M CHETHAN KESHAV BHAT	4VP23AI023
SKANDA PRASAD K	4VP23AI051
SRINIVAS HEGDE M	4VP23AI054

Under the Guidance of

Prof. Ajay Shastry C G

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt., of Karnataka

Nehru Nagar, Puttur - 574 203, DK, Karnataka, India.

November, 2026

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka

Nehru Nagar, Puttur - 574203, DK, Karnataka, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

Certified that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** is carried out by **Mr. K Shreekrishna Upadhyaya, Mr. M Chethan Keshav Bhat, Mr. Skanda Prasad K, Mr. Srinivas Hegde M** bearing USNs **4VP23AI020**, **4VP23AI023, 4VP23AI051 and 4VP23AI054** respectively bonafide students of **Vivekananda College of Engineering & Technology, Puttur** in partial fulfilment for the award of **Bachelor of Engineering** in **Artificial Intelligence & Machine Learning** of the **Visvesvaraya Technological University**, Belagavi during the year 2026-27. It is certified that all corrections/suggestions indicated during Internal Assessment have been incorporated in the report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of the Guide	Signature of the HOD	Signature of the Principal
Prof. Ajay Shastry C G	**Dr. Radhika Shetty D S**	**Dr. Purushothama Chippar**

EXTERNAL VIVA

Name of the Examiners	Signature with date
1…….....	.....
2…….....	.....

DECLARATION

We, **Mr. K Shreekrishna Upadhyaya (4VP23AI020), Mr. M Chethan Keshav Bhat (4VP23AI023), Mr. Skanda Prasad K (4VP23AI051), Mr. Srinivas Hegde M (4VP23AI054)** students of B.E. 7th Semester in Artificial Intelligence & Machine Learning, **Vivekananda College of Engineering & Technology**, Puttur, hereby declare that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** has been carried out by us at VCET, Puttur, under the guidance of **Prof. Ajay Shastry C G,** Assistant Professor, Department of Artificial Intelligence & Machine Learning, Vivekananda College of Engineering & Technology, Puttur, and submitted in partial fulfilment of the requirements for the award of degree in **Bachelor of Engineering in Artificial Intelligence & Machine Learning** by **Visvesvaraya Technological University**, Belagavi during the academic year 2026-2027.

Name of the students	USN	Signature with date
K Shreekrishna Upadhyaya	4VP23AI020	
M Chethan Keshav Bhat	4VP23AI023	
Skanda Prasad K	4VP23AI051	
Srinivas Hegde M	4VP23AI054	

Date:

Place: Puttur

ACKNOWLEDGEMENT

We take this opportunity to express our deep heartfelt gratitude to all those people who have helped us in the successful completion of the project.

First and foremost, we would like to express our sincere gratitude to our guide, **Prof. Ajay Shastry C G.** for providing excellent guidance, encouragement and inspiration throughout the project work. Without his invaluable guidance, this work would never have been a successful one.

We would like to express our sincere gratitude to Head of the Department of Artificial Intelligence & Machine Learning, **Dr. Radhika Shetty D S** for her guidance and inspiration.

We would like to thank our Principal, **Dr. Purushothama Chippar.** for providing all the facilities and a proper environment to work in the college campus.

We would like to thank our management for providing the necessary infrastructure to carry out the project work.

We are thankful to all the teaching and non-teaching staff members of Artificial Intelligence & Machine Learning Department for their help and needed support rendered throughout the project.

ABSTRACT

Stuttering disrupts the natural flow of communication through involuntary repetitions, prolongations, blocks, and pauses. Roughly 1% of people worldwide are affected, and the condition can reach into their social, professional, and personal life. Diagnosis today still relies mostly on manual assessment by speech-language pathologists, which is subjective, time-consuming, and not available to everyone. This project builds an automated system that uses deep learning to detect, classify, and analyze stuttering events in speech recordings. The system identifies the types of dysfluencies present and reports when they occur. Users record or upload audio through a simple interface and receive the results in an accessible format. The result is a tool for early stutter detection and objective speech evaluation, useful to both speech-language pathologists and people in treatment. The system also contributes to the growing use of artificial intelligence in healthcare, supporting diagnosis, therapy, and an improved quality of life for people who stutter.

TABLE OF CONTENTS

Title	Page No.
LIST OF TABLES	I
LIST OF FIGURES	II
LIST OF ABBREVIATIONS	III
CHAPTER 1 INTRODUCTION	**01**
1.1 INTRODUCTION TO THE PROJECT	01
1.2 PROPOSED METHOD	01
1.3 LITERATURE SURVEY	02
1.4 PROBLEM STATEMENT	12
1.5 OBJECTIVES	12
CHAPTER 2 REQUIREMENT SPECIFICATION AND ANALYSIS	**14**
2.1 INTRODUCTION	14
2.2 FUNCTIONAL REQUIREMENTS	14
2.2.1 User Requirements	14
2.2.2 System Requirements	15
2.3 NON-FUNCTIONAL REQUIREMENTS	15
2.3.1 Performance	15
2.3.2 Scalability	16
2.3.3 Availability	16
2.3.4 Usability	16
2.4 Software Requirements	16
2.5 Hardware Requirements	17
CHAPTER 3 SYSTEM DESIGN	**18**
3.1 INTRODUCTION	18
3.2 SYSTEM ARCHITECTURE AND DESIGN	18
3.3 FLOWCHART	21
3.4 USE CASE DIAGRAM AND DESCRIPTIONS	23
3.5 WORKFLOW DIAGRAM	25
3.6 SEQUENCE DIAGRAM	26
CHAPTER 4 SYSTEM IMPLEMENTATION	**28**
4.1 INTRODUCTION	28
4.2 ALGORITHM	28
4.3 IMPLEMENTATION REQUIREMENTS	29
4.3.1 Train the Model	30

4.3.2 Test the Model 30
4.3.3 Model Evaluation 31
4.4 USER INTERFACE 32
CHAPTER 5 SYSTEM TESTING 34
5.1 INTRODUCTION 34
5.2 TYPES OF TESTS 34
5.2.1 Unit Testing 34
5.2.2 Integration Testing 35
5.2.3 Functional Testing 35
5.2.4 System Testing 35
5.3 RESULT AND ANALYSIS 36
5.3.1 Model Evaluation of the Wav2Vec2 Classifiers 36
5.3.2 Wav2Vec2 Classifier Training 40
5.3.3 Model Evaluation of the CNN-based Models 41
5.3.4 CNN-based Model Training 42
5.3.5 Hyper Parameter Tuning 42
5.3.6 Performance Analysis of the Models 43
CHAPTER 6 SCREENSHOTS 45
CHAPTER 7 CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS 48
7.1 CONCLUSION 48
7.2 SCOPE FOR FUTURE ENHANCEMENTS 49
REFERENCES 50
PERSONAL PROFILE 53

LIST OF TABLES

Table No.	Title	Page No.
1.1	Summary of Literature Survey	07
5.1	Multitask Wav2Vec2 classifier results	36
5.2	Wav2Vec2 localizer evaluation results	39
5.3	Threshold optimization results	42
5.4	Comparative study of seven architecture variants	43

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	System Architecture	19
3.2	Flowchart	22
3.3	Use Case Diagram	24
3.4	Workflow Diagram	25
3.5	Sequence Diagram	26
5.1	Per-class F1 of the multitask Wav2Vec2 classifier	37
5.2	Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier	37
5.3	Confusion matrices for the five binary classifiers	38
5.4	Frame-level metrics of the Wav2Vec2 localizer	40
5.5	Per-class tuned F1 on the Boli set	43
6.1	Home page of the web interface	45
6.2	Results page of the web interface	45
6.3	Home page of the desktop application	46
6.4	Analysis result page of the desktop application	47

LIST OF ABBREVIATIONS

Abbreviation	Description
ASR	Automatic Speech Recognition
AUROC	Area Under the Receiver Operating Characteristic Curve
AUPRC	Area Under the Precision-Recall Curve
BERT	Bidirectional Encoder Representations from Transformers
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HMM	Hidden Markov Model
HuBERT	Hidden-Unit BERT
isWER	Intended Speech Word Error Rate
LLM	Large Language Model
LPC	Linear Predictive Coding
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
PCA	Principal Component Analysis
PWS	People Who Stutter
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SEP-28K	Stuttering Event Detection Dataset (28,000 samples)
SLP	Speech-Language Pathologist
SOTA	State of the Art
SVM	Support Vector Machine
TTS	Text-to-Speech
UAR	Unweighted Average Recall
UI	User Interface
VITS	Variational Inference with Adversarial Learning for end-to-end Text-to-Speech
WavLM	Wave Language Model
WER	Word Error Rate
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE PROJECT

Speech fluency lets a person express thoughts and feelings clearly. Stuttering interrupts that flow with involuntary repetitions, prolongations, or blocks. It affects people of all ages, and it can wear down confidence and make conversation and social contact harder. Spotting the patterns early makes therapy and day-to-day management easier.

In practice, detection and assessment still depend on a speech-language pathologist who listens to recordings and marks what they hear. Two clinicians can disagree about the same sample, the work is slow, and repeating it is tiring. That dependence on human listening also makes large-scale assessment and continuous monitoring hard to organise. Deep learning and speech signal processing now offer a scalable alternative. This project builds a system that detects, classifies, and analyzes five stuttering types: prolongations, blocks, sound repetitions, word repetitions, and interjections.

The pipeline preprocesses input audio, converts it into embeddings, and analyzes it with Wav2Vec 2.0, which captures fine acoustic detail and dependencies across longer stretches of speech. A second module localizes the stuttering events and returns their timestamps. Those timestamps are aligned with the transcript, so the system flags only the affected segments. The finished model sits behind a simple interface where a user records or uploads a sample and receives timestamped detections, class labels, and confidence scores.

The aim is an accurate, objective, and accessible diagnostic tool for speech-language pathologists and people who stutter. The work also adds to the use of artificial intelligence in healthcare, where the same analysis can support diagnosis, therapy, and a better quality of life.

1.2 PROPOSED METHOD

The proposed method is a deep learning system that detects and classifies speech dysfluencies early, using audio recordings as input. Preprocessing removes noise, normalizes loudness, and sharpens acoustic detail across all samples. Segmentation isolates the regions that carry speech so silence and background noise drop out. Augmentation such as noise injection, time and pitch shifting, and amplitude scaling exposes the model to different speaking rates and recording conditions. The system then assigns each input

to one of five dysfluency types and marks where the event occurred, which supports early diagnosis and treatment planning. Automating this work cuts manual effort, reduces observer bias, and makes the assessment more reliable.

1.3 LITERATURE SURVEY

P. Arbajian et al. [1] The paper studies how the choice of speech segment affects stutter block detection. The authors extract acoustic features from annotated samples and compare classifiers under different segmentation setups, including several windowing approaches. Accuracy depends strongly on segment length: some lengths capture dysfluency patterns better, while poor segmentation drops temporal information and performance falls. The finding shaped the preprocessing stage of this project.

V. Mitra et al. [2] This study tunes an existing hybrid ASR system so a voice assistant handles dysfluent speech better. The aim is to cut recognition errors caused by repetitions and unintended insertions. The method changes decoding parameters: it raises the penalty for inserted words and lowers the weight of the acoustic model, so the language model filters repeated or disfluent segments and the recogniser focuses on meaningful speech. On recordings from 18 participants with different stuttering severity, intended speech word error rate fell by 24% relative. Parameter tuning alone achieved that gain without rebuilding the ASR architecture.

P. Mohapatra et al. [3] The paper introduces disfluencyNet, a model that detects speech disfluencies from contextual representations. Contextual embeddings feed a classification network that models dependencies in speech and catches repetitions and pauses. Data distillation reduces the need for a large training set. On SEP-28k and FluencyBank, the model matched baseline accuracy with a quarter of the data. That result supports the contextual embeddings and efficient training used in this project.

J. Liu et al. [4] This paper proposes a method for detecting disfluencies in multilingual speech of different lengths, using the contextual embeddings of wav2vec 2.0. A classification network takes the embeddings, and data distillation keeps only audio fragments where three human annotators agree on the label. Tests on multilingual datasets with varied speech lengths beat other approaches. The work supports the use of pretrained models and data filtering for robustness across speakers.

A. Romana et al. [5] The study detects disfluencies directly from untranscribed audio. A frame-level Bi-LSTM fusion model combines WavLM acoustic features with BERT

representations taken from transcripts produced by a fine-tuned Whisper model. The combination captures low-level speech patterns and higher-level context, and it tolerates ASR errors and misalignment. In noisy real-world conditions the multimodal setup was more robust and accurate than either modality alone.

D. Wagner et al. [6] This paper uses large language models to combine acoustic and linguistic representations for dysfluency detection. Acoustic features come from wav2vec 2.0 and transcriptions from Whisper; the two are fused into a joint input that an LLM classifies into stutter types, including repetitions, prolongations, and blocks. The acoustic-lexical combination beat models that used only audio or only text. That supports the plan in this project to combine audio and language signals.

S. A. Sheikh et al. [7] The study develops multi-contextual StutterNet, a framework that addresses class imbalance and limited data. A multi-branch architecture processes several contextual representations of the same speech. Class-balanced loss gives more weight to underrepresented stutter categories, and augmentation diversifies the training set. Detection was more robust across speech conditions, especially for less frequent dysfluency types. This project borrows the same three ideas: augmentation, balanced loss, and multi-context learning.

X. Zhou et al. [8] The paper treats spectrograms as images and adapts a YOLO-style detector to find dysfluencies in the time-frequency domain. A region-based model learns frequency patterns and changes over time, and speech-text alignment links audio segments to their words for context. The model predicts both the dysfluency type and the time region where it occurs, with strong results across several types. This project follows that region-wise, spectrogram-based approach.

X. Zhou et al. [9] Stutter-Solver is an end-to-end, YOLO-inspired model for multilingual dysfluency detection. It treats speech spectrograms as image-like inputs and applies a region-based detector. To cope with scarce data, the authors generated synthetic datasets (VCTK-Pro, VCTK-Art, and AISHELL3-Pro) with articulatory and text-to-speech simulations. The model performed well across datasets and languages, which supports the multilingual aims and synthetic data plans in this project.

J. Zhang et al. [10] This paper proposes LLM-Dys, a method for generating large dysfluent speech datasets with large language models. An LLM simulates realistic dysfluency patterns and writes labelled dysfluent text, which VITS converts into audio. Compared

with rule-based generation, the speech keeps more natural prosody and contextual variation. The dataset covers 11 dysfluency categories at word and phoneme level, which allows fine-grained training and analysis. Models trained on it performed better on detection tasks.

S. Kim and A. Kumar [11] FluentNet is a CNN-LSTM architecture for end-to-end speech disfluency detection. CNN layers extract short-term spectral features from mel-spectrograms, and LSTM layers model temporal continuity, which suits recurring stutter patterns. On SEP-28k it reached 91% classification accuracy and ran in real time in therapy, where users received visual feedback. Skipping handcrafted features reduced bias. The architecture informs the CNN models compared in this project.

R. Ahmed and J. Park [12] This study builds a multilingual framework that detects stuttering events in English, Korean, and Japanese data. The model uses an encoder-decoder transformer similar to BERT and captures contextual relationships in speech sequences. Trained on the combined multilingual datasets, it adapted well to linguistic variation and reached an F1-score of 95.2% in English and over 93% on non-English corpora. Attention visualisation showed which input segments influenced each decision.

F. Rahimi and D. Torres [13] This paper explores large language models and transformer architectures for speech dysfluency analysis. Speech features are embedded as tokenized sequences so the model can detect contextual disruptions. The LLM approach beat CNN and RNN models in recall and precision, particularly for subtle types such as interjections and soft blocks. Pretraining on general speech data also helped on smaller stutter-specific corpora, and attention visualisation made predictions easier to inspect.

V. Uloza et al. [14] This paper develops a deep neural network for assessing substitution voicing in pathological speech. CNNs and principal component analysis handle classification, and preprocessing such as noise removal and normalization gives the networks consistent input. Accuracy exceeded 93%, which shows that AI can detect subtle speech impairments. The study is about substitution voicing rather than stuttering, but its use of spectral analysis and CNN features informed this project.

H. Müller and C. Lee [15] This meta-analysis of fMRI and EEG studies looks at the neural bases of speech production in people who stutter. The inferior frontal gyrus and basal ganglia show atypical activation during speech. Those findings give a biological grounding to acoustic detection, since the irregularities a model picks up reflect differ-

ences in neural control. The study proposes no computational model, but it helps explain what the detected events represent.

A. Baevski et al. [16] wav2vec 2.0 learns speech representations directly from raw audio in a self-supervised way. A convolutional encoder extracts latent features and a transformer adds context. Pretrained on 960 hours of unlabelled LibriSpeech and fine-tuned with a CTC loss, it performed strongly on phoneme recognition, speaker identification, and emotion recognition. It is the backbone of this project, and its pretraining is what makes feature extraction possible without a large labelled stutter corpus.

P. Khanna et al. [17] StuD is a multimodal stuttering detector that combines Wav2Vec 2.0 and HuBERT acoustic features with Llama-2 linguistic features and retrieval-augmented generation. A fusion strategy weights the acoustic and linguistic embeddings by input quality, and the retrieval component pulls similar historical cases to classify rare patterns. On SEP-28k and FluencyBank it performed strongly across event types. The fusion design supports the combined audio and text analysis in this project.

C. Lea and V. Mitra [18] SEP-28K contains over 28,000 clips from stuttering support group podcasts, annotated for blocks, prolongations, repetitions, interjections, and revisions. Clips carry crowdsourced labels from non-expert listeners and expert labels from speech-language pathologists. Scaling annotation from 10k to 28k clips improved F1 by 28% for blocks and 24% for prolongations. It is the main benchmark for this project.

O. Shonibare et al. [19] This paper proposes a two-stage approach to ASR on stuttered speech with limited labelled data. A context-aware classifier trained on a small amount of labelled stuttering data finds dysfluency regions, and a modified ASR decoder passes over them so transcription focuses on fluent segments. Word error rate fell by 12% to 71% across severity levels. The detect-then-transcribe design shows that detection and speech processing can be separated, which is how this project's pipeline is organised.

S. Bayerl et al. [20] The study applies a fine-tuned wav2vec 2.0 to dysfluency detection in clinical therapy recordings. Acoustic embeddings go through a multi-task framework and an SVM classifier that separates dysfluency types. On FluencyBank and the German KSoF dataset, F1 improved by 27% over baselines. Predicting presence and type together lets therapists track progress across sessions, which shows that pretrained speech models transfer to clinical assessment.

S. Bayerl et al. [21] This paper treats stuttering detection as a multi-label problem, since dysfluencies often occur together. A modified wav2vec 2.0 assigns an independent probability to each type, so overlapping events are detected at once. It performed well on SEP-28k-Extended and transferred from English to German. The formulation matches clinical reality: one utterance can contain a block, a prolongation, and repetitions together.

R. Gong et al. [22] AS-70 is the first large Mandarin stuttered speech dataset, with 70 hours from speakers at different severity levels. It covers age groups, genders, and stuttering types, and includes word- and syllable-level dysfluency labels. The corpus is open source. It extends stuttering data beyond English and makes multilingual systems possible.

X. Liu et al. [23] This paper combines Conformer blocks for local acoustic patterns with BiLSTM layers for long-range temporal dependencies. The multi-task framework predicts dysfluency type and severity together. It took first place in the SLT 2024 Stuttering Detection Challenge and improved F1 by 39.8% on AS-70. Conformer's mix of convolution and self-attention captures local and global speech patterns, which suits the varied timing of stuttering events.

A. Batra et al. [24] Boli is a multilingual Indian language dataset with read and spontaneous speech from people who stutter. It covers five stuttering types, records demographic metadata, and includes self-reported severity ratings. The corpus fills a gap in non-English stuttering data, especially for South Asian languages, where prevalence and manifestation differ from Western populations.

A. R. Valente et al. [25] This study adds detailed clinical annotations from expert speech-language pathologists to FluencyBank. The labels cover audiovisual cues and secondary behaviours such as facial tension and eye blinking, plus physiological tension indicators, beyond the usual acoustic labels. The multi-dimensional scheme captures more of the clinical picture of severity. Such annotations allow models to account for the full stuttering experience rather than acoustic events alone.

T. Grósz et al. [26] This paper presents a wav2vec2-based system that recognises vocalised emotions and stuttering events together. Self-supervised speech embeddings feed task-specific classification heads for both tasks. It reached 62.1% unweighted average recall on the Stuttering Sub-Challenge. The results connect emotional state and disfluency and suggest that shared representations help detection.

J. Tang et al. [27] This paper presents annotation guidelines developed with people who stutter, standardising how stuttering events are labelled. The guidelines resolve ambiguous cases with categorical definitions and decision trees, and they cover annotator training, inter-annotator agreement, and edge cases. Community-informed guidelines raise annotation quality and reproducibility, which helps dataset construction and model evaluation.

A. Romana et al. [28] The study releases an updated FluencyBank with word-level timestamps and refined disfluency annotations. Transcripts carry granular labels and word timing for each clip, so models can be measured on disfluent input. The resource supports word-level localization, which is what this project needs to map events to the words where they occur.

L. Nie et al. [29] MMSD-Net is the first multimodal neural framework for stuttering detection, combining audio and visual signals through transformer-based cross-modal fusion. It processes speech embeddings alongside facial expression and lip movement features. F1 improved by 2% to 17% over single-modality approaches on benchmark datasets. Visual cues help most for events such as blocks and prolongations that have distinct visual signatures.

R. P. Buzzeti et al. [30] This paper proposes a two-stage hybrid method for detecting and classifying stuttering-related disfluencies in Brazilian Portuguese. The first stage finds possible disfluency regions from linguistic rules, and the second assesses severity. Portuguese stuttering differs from English in syllable structure and in how dysfluencies surface, and the method addresses those patterns. It extends detection to an underrepresented language.

The studies cover acoustic classifiers, self-supervised speech representations, multimodal fusion, and synthetic data. Table 1.1 compares them by features, strengths, and limitations. The field has moved from hand-crafted acoustic features toward large pretrained models. Multilingual coverage, localization, and clinical reliability remain open.

Sl. No	Author	Title	Features	Pros	Cons
1	Pierre Arbajian et al. [1]	Effect of speech segment samples selection in stutter	Analyzed different speech segment lengths and sample selection meth-	Improves detection precision through better segment selection.	Sensitive to segmentation configuration.

Sl. No	Author	Title	Features	Pros	Cons
		block detection and remediation	ods for accurate stutter block detection using acoustic classifiers.		
2	Vikramjit Mitra et al. [2]	Analysis and Tuning of a Voice Assistant System for Dysfluent Speech	Modified ASR decoding parameters by increasing word insertion penalty and reducing acoustic model influence to suppress repetition errors	Enhances intended speech recognition for dysfluent users.	Focuses on recognition rather than dysfluency classification.
3	Payal Mohapatra et al. [3]	Speech Disfluency Detection with Contextual Representation and Data Distillation	Developed DisfluencyNet using contextual embeddings and distilled high-confidence samples for efficient low-resource training	Achieves strong results with reduced training data	Depends on carefully filtered annotations
4	Jiajun Liu et al. [4]	Automatic Speech Disfluency Detection Using Wav2Vec 2.0 for Different Languages	Applied Wav2Vec 2.0 contextual embeddings for multilingual disfluency detection across variable speech durations.	Supports multilingual detection with strong contextual learning.	High resource requirements and limited availability of multilingual stuttering datasets.
5	Amrit Romana et al. [5]	Automatic Disfluency Detection from Untranscribed Speech	Built multimodal Bi-LSTM combining WavLM acoustic signals with BERT linguistic features from Whisper transcripts.	Detects disfluencies without manual transcription.	Multimodal design increases system complexity.
6	Dominik Wagner et al. [6]	Large Language Models for Dysfluency Detection in Stuttered Speech	Combined Wave2Vec 2.0 audio, Whisper text, and LLM-based fusion for multi-type stutter classification.	Improves multi-class detection accuracy.	Requires heavy computational resources.
7	Shakeel A. Sheikh et al. [7]	Advancing Stutter Detection via Data Augmenta-	Proposed multi-contextual StutterNet with augmentation	Addresses data scarcity and class imbalance.	Synthetic augmentation may reduce realism.

Sl. No	Author	Title	Features	Pros	Cons
		tion, Class-Balanced Loss and Multi-Contextual Deep Learning	and weighted loss for robust stuttering detection.		
8	X. Zhou et al. [8]	YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection	Adapted YOLO on speech spectrograms for simultaneous dysfluency type prediction and temporal localization.	Enables precise real-time stutter localization.	Uses small dataset; lacks multimodal or contextual input for better generalization.
9	Xuanru Zhou et al. [9]	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Developed multilingual YOLO-based dysfluency detector using synthetic articulatory and TTS-generated datasets.	Expands multilingual coverage with SOTA accuracy.	Synthetic speech may not fully match real speech.
10	Jinning Zhang et al. [10]	Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection	Proposed LLM-Dys framework using LLM-generated dysfluent text and VITS TTS for scalable corpus creation.	Generates diverse large-scale dysfluency datasets.	Synthetic prosody may limit robustness.
11	S. Kim & A. Kumar [11]	FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning	CNN-LSTM hybrid model analyzing temporal and spectral dependencies in speech signals using SEP-28k dataset.	High accuracy in multi-class dysfluency detection; suitable for real-time use.	Model complexity increases training time and requires GPU-based systems.
12	R. Ahmed & J Park [12]	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Transformer-based multilingual model for detecting stuttering across multiple languages using contextual embeddings.	Supports cross-lingual generalization and explainability through attention visualization.	High resource requirements and limited availability of multilingual stuttering datasets.
13	F. Rahimi & D. Torres [13]	Large Language Models for Dysfluency Detection in Stuttered Speech	Used transformer-based large language models (LLMs) to capture context disruptions in stuttered speech	Outperforms CNN and RNN baselines on subtle dysfluency types.	Requires heavy computational resources.
14	V. Uloza et al. [14]	An AI-Based Algorithm for the Assess-	CNN and PCA combine feature extraction and classifica-	Demonstrates effectiveness of AI in medical speech	Focused on substitution voicing, not directly stuttering-re-

Sl. No	Author	Title	Features	Pros	Cons
		ment of Substitution Voicing	tion of pathological voice disorders.	analysis; high accuracy (>93%).	lated; limited dataset scope.
15	H Muller & C. Lee [15]	Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis	Analyzed fMRI/ EEG studies to identify brain regions linked to speech dysfluency.	Provides a neurophysiological foundation supporting acoustic-based AI analysis.	Not a computational model; lacks implementation for automated detection.
16	A. Baevski et al. [16]	wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations	Self-supervised pretraining of speech representations using masked prediction with transformer and convolutional encoder on raw audio.	Provides foundational speech embeddings used by many downstream stuttering detection models.	Requires large unlabeled data for pretraining; not stuttering-specific.
17	P. Khanna et al. [17]	StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies	Combined Wav2Vec 2.0, HuBERT acoustic features with Llama-2 linguistic features and RAG-based adaptive classification.	Achieves SOTA on SEP-28k and FluencyBank with adaptive retrieval.	Heavy computational requirements for LLM and RAG components.
18	C. Lea and V. Mitra [18]	SEP-28K: A Dataset for Stuttering Event Detection from Podcasts	Large-scale dataset with 28k+ clips annotated for 5 stuttering event types from podcast recordings.	Enables large-scale training with 28%/24% F1 gains from data scaling.	Podcast speech may not generalize to clinical or spontaneous settings.
19	O. Shonibare et al. [19]	Enhancing ASR for Stuttered Speech with Limited Data using Detect and Pass	Two-stage approach: context-aware dysfluency detection followed by ASR that passes over detected events.	Achieves 12-71% WER reduction with minimal labeled data.	Relies on accurate first-stage detection; cascaded errors possible.
20	S. Bayerl et al. [20]	Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0	Fine-tuned wav2vec 2.0 with multi-task learning and SVM for therapy speech analysis.	27% F1 improvement on clinical therapy recordings.	Limited to therapy contexts; may not generalize to casual speech.
21	S. Bayerl et al. [21]	Dysfluencies Seldom Come Alone - Detection as a	Modified wav2vec 2.0 for simultaneous multi-label detection	SOTA on SEP-28k-Extended with cross-language generalization.	Multi-label training increases model complexity and an-

Sl. No	Author	Title	Features	Pros	Cons
		Multi-Label Problem	of co-occurring dysfluencies.		notation requirements.
22	R. Gong et al. [22]	AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition	First large-scale Mandarin stuttered speech dataset with verbatim transcriptions and detailed annotations.	Enables Mandarin stuttering research with 70h of diverse speaker data.	Limited to Mandarin; annotation process labor-intensive.
23	X. Liu et al. [23]	An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM	Conformer blocks with BiLSTM temporal modeling for multi-task dysfluency and severity prediction.	1st place SLT 2024 Challenge; 39.8% F1 improvement on AS-70.	Conformer architecture requires significant GPU memory and training time.
24	A. Batra et al. [24]	Boli: A Dataset for Understanding Stuttering Experience	Multi-lingual Indian language dataset with read and spontaneous speech across 5 stutter types.	Captures real-world Indian stuttering patterns across multiple languages.	Limited annotations; may need expert review for clinical applications.
25	A. R. Valente et al. [25]	Clinical Annotations for Automatic Stuttering Severity Assessment	Enhanced FluencyBank with expert clinician annotations including audiovisual and behavioral cues.	Provides clinical-grade multi-dimensional annotations for severity assessment.	Expert annotation is expensive and time-consuming to produce.
26	T. Grósz et al. [26]	Wav2vec2-based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering	Joint wav2vec2 framework for emotion recognition and stuttering detection from speech embeddings.	Demonstrates emotion-disfluency interconnection with 62.1% UAR.	Joint modeling may introduce task interference; limited to paralinguistic analysis.
27	J. Tang et al. [27]	Speech Annotation Guidelines with People Who Stutter	Standardized annotation protocols developed collaboratively with PWS for consistent dysfluency labeling.	Improves annotation quality and reproducibility across stuttering research.	Guidelines adoption requires community-wide coordination and training.
28	A. Romana et al. [28]	FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic	Updated FluencyBank with word-level timestamps and refined disfluency annotations	Enables word-level dysfluency localization and more accurate model evaluation.	Limited to English stuttering patterns; requires updated annotation protocols.

Sl. No	Author	Title	Features	Pros	Cons
		Intended Speech Recognition	for precise temporal alignment.		
29	L. Nie et al. [29]	MMSD-Net: Towards Multi-modal Stuttering Detection	First multi-modal neural framework combining audio and visual signals through transformer-based cross-modal fusion.	2-17% F1 improvement over uni-modal approaches; captures visual stuttering cues.	Requires video input; increased computational complexity and data collection overhead.
30	R. P. Buzzeti et al. [30]	Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese	Two-stage hybrid approach combining rule-based detection with ML classification for Portuguese stuttering.	Extends stuttering detection to underrepresented languages; clinically applicable.	Language-specific rules may not transfer to other languages without modification.

Table 1.1: Summary of Literature Survey

1.4 PROBLEM STATEMENT

Stuttering assessment in clinical practice still depends heavily on a speech-language pathologist who listens to recordings and marks dysfluencies by hand. The process is slow, subjective, and difficult to reproduce, and disagreement between raters makes a patient's progress hard to track. Most automated tools stop at binary detection: they report that a recording contains stuttering but not which type occurred or where in the audio it occurred, which limits their use for therapy planning.

This project addresses that gap with a single pipeline that takes an unconstrained speech recording and produces a structured analysis: whether stuttering is present, which of the five dysfluency types are involved, and the exact time regions where each event occurs. The system must also work without a large annotated corpus from the target speaker and must run fast enough for interactive assessment rather than offline batch processing.

1.5 OBJECTIVES

The project is organised around four objectives, and each one supplies a stage of the final pipeline.

1. **Speech preprocessing:** clean and standardise the input audio so every downstream model receives consistent data. This covers resampling to a fixed rate, DC offset removal, peak normalization, silence trimming, fixed-length padding, and augmentation such as noise injection and time or pitch shifting, which together reduce the effect of varying recording conditions and speaker loudness.
2. **Stutter detection:** decide whether a sample contains dysfluent speech at all. The binary output gates the rest of the analysis, so the model is trained to hold up across speakers, languages, and background noise, not only on clean studio recordings.
3. **Stutter classification:** identify which dysfluency types are present among prolongation, block, sound repetition, word repetition, and interjection. Because these events often co-occur, the system decides each type independently and gives it its own confidence score rather than forcing one mutually exclusive label.
4. **Stutter localization:** find the time regions where detected events occur and align them with the corresponding words in the transcript. Clinicians get usable timestamps, so they can go straight to the affected segment and compare the detected events with what was said.

The objectives are ordered so each stage builds on the one before it. Preprocessing produces a normalized waveform and spectrogram for detection and classification. Detection narrows the analysis to samples that contain dysfluencies. Classification assigns type-level scores, and localization maps the events back onto the time axis and the transcript. Together these stages turn a recording into evidence that can be inspected at clip level and at event level. The classification scores summarise what was detected, while the localized intervals and aligned transcript show where it happened, which makes the result easier to review.

CHAPTER 2

REQUIREMENT SPECIFICATION AND ANALYSIS

2.1 INTRODUCTION

A deep learning framework for stuttering classification and localization has to satisfy two sets of expectations at once, clinical and technical. Before implementation, the project therefore fixes its functional and non-functional requirements, the system components, and the workflow that connects them. The purpose is to replace manual speech-language assessment with a model that detects dysfluencies early and reports them objectively.

The system takes an audio recording, extracts features, and classifies them into the five stuttering types. It also locates each event in the signal. That should cut the time manual analysis takes and reduce variation between observers, without making the output hard to read. Wav2Vec 2.0 handles feature extraction, a Wav2Vec2 frame-level model handles localization, and Whisper produces the transcript. The system must run on standard hardware, cope with larger datasets as they arrive, offer a simple interface, and keep patient data secure and confidential.

2.2 FUNCTIONAL REQUIREMENTS

The functional requirements describe what the system must do to detect and classify stuttering events. Clinicians, speech-language pathologists, and people who stutter can upload a recording or capture audio through the interface. The supported formats are WAV, MP3, FLAC, and M4A.

The system processes the audio and sorts it into five dysfluency types: prolongation, block, sound repetition, word repetition, and interjection. For each detected class, the interface shows the result with a confidence score, so the user can judge how much to trust the prediction. The system also returns the timestamps of localized events. Waveforms, spectrograms, and overlays mark the regions that influenced the model's decision. Users can download the full analysis report, and uploaded audio and results are stored securely for later reference.

2.2.1 User Requirements

User requirements describe what the end users need to be able to do. They reflect how clinicians, speech-language pathologists, researchers, and people who stutter will use the system while assessing or monitoring fluency. Those users need an interface that is simple

to navigate, that shows classification results, visual interpretations, and confidence levels without technical complexity, and that works with minimal manual effort. For a speech analysis platform, the general requirements are:

- The user should be able to upload or record a speech sample for analysis.
- The user should be able to see classification results as soon as the analysis finishes.
- The system should provide detailed analysis reports.

2.2.2 System Requirements

System requirements cover the software and hardware capabilities the system needs. The stuttering detection system has to handle the computational load of deep learning speech analysis while delivering the operations described above. Hardware guidance appears later in this chapter.

These requirements include:

- The system must accept audio uploads securely and classify them.
- Users need a way to save previous analysis results and open them again.
- Adequate storage must be available for managing large speech datasets.

2.3 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements set the quality, performance, and usability standards the system has to meet. Performance matters most, because the models run deep learning and audio processing and must return results quickly. GPU acceleration and memory management keep speech recordings moving through the pipeline. Reliability covers consistent accuracy across datasets and fault tolerance when a run is interrupted. Scalability allows larger datasets or extra dysfluency types without a drop in performance. Usability means a simple interface for technical and non-technical users alike, with clear upload, analysis, and visualisation steps. Maintainability depends on modular code that is easy to update and retrain. Portability and availability keep the system running across platforms, with high uptime and regular backups.

2.3.1 Performance

- The system should process and analyse a speech sample within 3 to 5 seconds.
- The classification results should keep false negatives and false positives low.
- Performance metrics should be monitored regularly.

- The system should handle audio uploads without significant delay.

2.3.2 Scalability

- The system should handle growing speech datasets efficiently.
- It should scale across multiple clinics and research centres.
- The system should adapt to improvements in deep learning models.
- The system should handle a higher volume of concurrent users.

2.3.3 Availability

- The system should stay accessible at all times, with minimal downtime during clinical use.
- Components should tolerate faults so a single failure has less impact.
- The platform should keep serving users during updates or model improvements.

2.3.4 Usability

- The interface should be easy for clinicians, speech-language pathologists, and non-technical users.
- Audio upload and results should be easy to find and navigate.
- The visualisation outputs should help users understand the model's predictions for the analysed speech.

2.4 Software Requirements

The software requirements cover the tools used to develop, train, validate, and deploy the stutter detection models.

- **Operating System:** the system runs on Linux and Windows, including Windows 10 and above and common Linux distributions. The PySide6 desktop application and the web application run on either platform, and training and inference are supported on CPU or an NVIDIA GPU.
- **Deep Learning Framework:** PyTorch is the primary framework for building and training the models. It provides GPU support through torch.compile, mixed precision, and TensorFloat-32 (TF32), and it scales to the audio classification tasks in this project. Hugging Face Transformers supplies the pretrained speech models, Wav2Vec 2.0 for feature extraction and Whisper for timestamped transcription.

- **Libraries:** a range of Python libraries supports different stages of the project:
    - librosa: Used for audio preprocessing and manipulation, including loading, resampling, mel-spectrogram computation, and silence trimming.
    - NumPy: Provides support for numerical operations and array handling.
    - Pandas: managing structured data such as labels or metadata.
    - Scikit-learn: Used for data splitting, model evaluation metrics, and traditional machine learning algorithms.

2.5 Hardware Requirements

The hardware requirements give the minimum and recommended specifications for running the models.

- **Processor (CPU):** a multi-core processor such as an Intel Core i5 (8th generation or newer) or AMD Ryzen 5 and above. It provides enough power for audio preprocessing and training workflows.
- **Storage (Hard Disk):** at least 10 GB of free disk space for datasets, checkpoints, logs, and temporary files. An SSD is recommended for faster read and write speeds.
- **Memory (RAM):** at least 8 GB of RAM to run the training scripts and handle datasets, with 16 GB or more recommended during training and validation, when data loading and augmentation are active.
- **Graphics Processing Unit (GPU):** an NVIDIA GPU with CUDA support is recommended for faster training. Cards such as the RTX 2060 or higher (RTX 3060, RTX 3080) cut training time and handle larger datasets and deeper networks. CUDA and cuDNN must be installed for full compatibility with PyTorch.
- **Audio Input Device:** a working microphone is required for recording speech in the application.

The minimum configuration handles inference on short clips. Fine-tuning the larger pretrained backbones benefits from the recommended configuration, and a GPU with at least 8 GB of memory is preferable, though classification-only mode runs comfortably on a CPU.

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

System design is an essential stage in software development that focuses on defining the overall structure and functionality of a system. It translates user and technical requirements into a detailed framework that outlines how different components, modules, and interfaces will interact. The process involves creating both high-level architecture, which defines the system's major elements and their interconnections, and low-level design, which specifies detailed components such as data structures, algorithms, and communication flows.

The main objective of system design is to build a solution that fulfills user needs while ensuring performance, reliability, scalability, and security. It also emphasizes maintainability and flexibility so the system can adapt to future enhancements. Effective system design requires collaboration among developers, architects, and domain experts to ensure the architecture aligns with functional goals and business requirements. By carefully organizing how each part of the system operates and communicates, system design establishes a strong foundation for efficient implementation and long-term stability.

3.2 SYSTEM ARCHITECTURE AND DESIGN

The system architecture defines the overall structure and organization of the Stuttering Classification and Localization System using deep learning. It provides a comprehensive framework describing how different components interact to process speech audio, extract discriminative features, and classify utterances into normal and dysfluent categories. A well-structured architecture ensures efficient data flow, modularity, and scalability, allowing the model to deliver high diagnostic accuracy while remaining adaptable to future advancements. The architecture covers the main stages of the pipeline, from preprocessing and feature extraction to model training and classification, so that performance remains stable. Furthermore, it supports seamless integration with user interfaces for audio upload, visualization, and prediction display. Emphasis is placed on maintaining data security and patient confidentiality, ensuring ethical and reliable operation throughout the analysis workflow. Additionally, the architecture allows for performance optimization, enabling fast and accurate stutter detection even when deployed on large speech datasets.

The proposed deep learning architecture for the automatic classification and localization of stuttering using speech audio is illustrated in Figure 3.1. The framework integrates two model pathways, five independent Wav2Vec 2.0 binary classifiers for classification and a Wav2Vec2 frame-level pipeline for localization, to improve feature extraction, multi-perspective learning, and classification robustness. The complete workflow consists of five major phases: Input, Pre-processing, Feature Extraction, Classification, and Output. This dual-model structure enables richer representation learning and enhances overall detection accuracy. The architecture therefore separates reusable signal preparation from task-specific prediction. The same normalized waveform can be passed to classification, transcription, and localization, while the final fusion stage combines their outputs without duplicating preprocessing work.

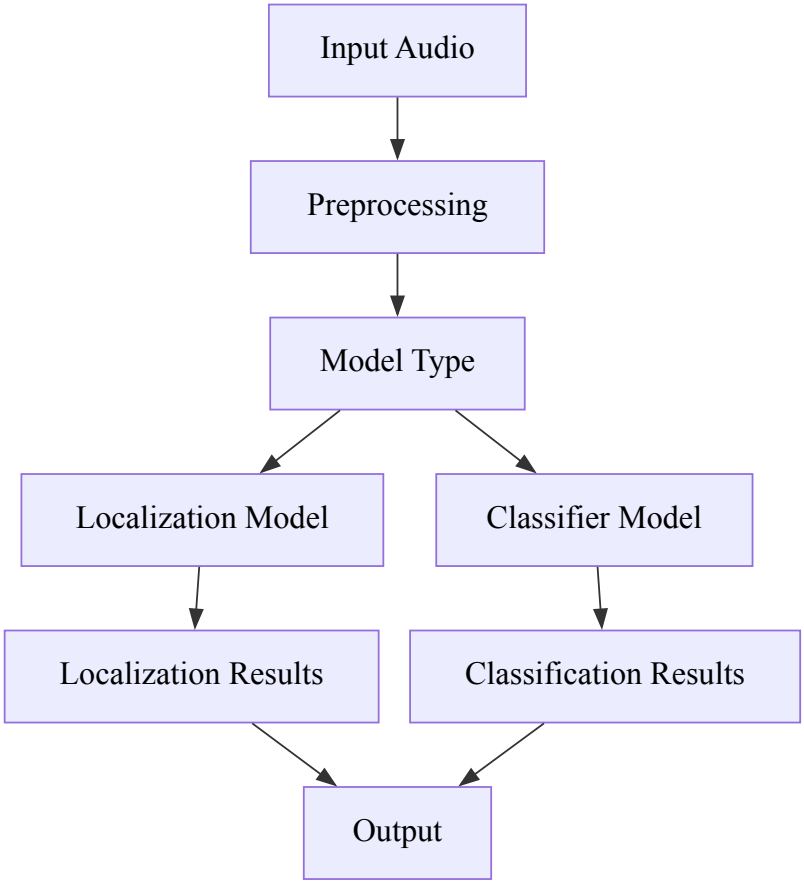


Figure 3.1: System Architecture

Speech Audio Input: The system begins with the collection of speech audio samples, which serve as the primary dataset for stutter detection. Audio enters the system when the user records directly or uploads a pre-recorded file through the web interface or desktop application. The web app captures audio with MediaRecorder APIs; the desktop app uses sounddevice. Each sample may contain one or more dysfluencies, including blocks, prolongations, word repetitions, sound repetitions, and interjections. Ensuring adequate recording quality is crucial, as variations in microphone hardware, background noise, and

speaking rate can affect model reliability. The dataset includes recordings from Project Boli, SEP-28K, and UCLASS, and the ingestion pipeline resolves the English material of all three sources. This stage gives the diagnostic system its high-quality data for the steps that follow.

Preprocessing: Before being fed into the deep learning model, speech audio undergoes preprocessing to enhance clarity, reduce noise, and ensure consistency across samples. The audio is sent to the backend and converted to 16 kHz mono WAV with FFmpeg so every clip is handled the same way. The preprocessing pipeline also includes DC offset removal, peak normalization, and silence trimming to clean the signal and remove unwanted artifacts. Fixed-length padding truncates or pads each clip to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length. After this step the input audio is standardized and optimized, so the deep learning architecture can extract features and classify it accurately.

Wav2Vec 2.0: The detection framework starts with Wav2Vec 2.0, a deep self-supervised model pre-trained on large amounts of unlabeled speech. Wav2Vec 2.0 quantizes the raw waveform into contextualized speech representations, learning robust acoustic features without requiring manually labeled data. This structure helps the model retain important information and capture fine-grained phonetic and prosodic patterns, such as broken phonation, repetitions, and elongated sounds, which are key indicators of dysfluent speech. Because it is pre-trained on large speech corpora, the model already understands general speech features, so adapting it to stutter detection with limited labeled data is easier. Because it extracts contextual features at several levels, it gives a strong base for detecting stutter events with high precision.

Detection and Localization Pipelines: To achieve higher accuracy, the proposed system uses a hybrid detection framework that combines classification and localization. After preprocessing, the audio runs through three parallel pipelines: classification, transcription, and localization. The classification pipeline extracts speech representations with Wav2Vec 2.0 and passes them through five independent binary classifiers, each trained for a distinct dysfluency type. The localization pipeline identifies dysfluency regions at the frame level using Wav2Vec2 frame-level feature extraction, so it can mark the precise segments of speech where dysfluencies occur. Combining the two approaches lets the system catch fine acoustic detail and broader prosodic structure, so results stay stable when recordings vary in noise, clarity, or speaking style.

Classification: Once feature extraction finishes, the Wav2Vec 2.0 representations go to the classification stage. Here, five independent binary classifiers assign a presence proba-

bility to each dysfluency type, using decision boundaries learned during training to separate fluent from dysfluent speech. Sigmoid outputs give per-class probabilities, and binary cross-entropy loss drives optimization during training. Their outputs are aggregated into a multi-label result that reports the presence probability of each dysfluency type and summarizes the detected classes and the primary dysfluency. A multitask shared-backbone variant was also trained for comparison, and its outputs are combined with localization at prediction time: the combiner labels each localized region with per-class saliency scores and fuses the two views into a single annotated result. The output gives speech-language pathologists diagnostic results they can act on when identifying stutter events.

Transcription and Alignment: The transcription pipeline uses the Whisper Automatic Speech Recognition (ASR) model to produce a timestamped transcript, with support for English, Kannada, and Hindi. A Connectionist Temporal Classification (CTC) based time alignment step ties the pipelines together, mapping the detected dysfluency regions to specific words or syllables in the transcript. Language-specific adapters keep the alignment accurate for English, Kannada, and Hindi.

Output: In the final stage, the system presents its predictions through a user-friendly interface. The output typically shows the detected dysfluency types along with confidence scores that indicate how certain the model is about each decision, and a summary of the primary dysfluency. To make the results more interpretable, the outputs surface as waveform overlays, spectrogram visualizations, timestamped transcripts, and a detailed clinical-style report. The outputs are designed to be simple, clear, and clinically relevant, making them suitable for practical use in clinical or self-assessment settings. By providing accurate and interpretable results, the system supports early assessment and can play a valuable role in improving outcomes for people who stutter.

Both the web and desktop applications load trained models through the centralized model registry. The registry reads from a registry module (the model/registry package) and a configuration file (registry.json), so checkpoints can be updated or replaced without changing application code, which keeps the system flexible and easy to maintain.

3.3 FLOWCHART

The process begins with the input audio, recorded or uploaded by the user. The audio is converted to 16 kHz mono WAV with FFmpeg, then padded or truncated to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length.

After normalization, the audio runs through classification, transcription, and localization in parallel.

A CTC-based alignment step combines these outputs and maps the detected dysfluencies to specific words or syllables. The final data is turned into per-word annotations, shown in waveform and spectrogram displays, and written into the analysis report. The stages stay separate so each can be tested on its own, and every branch receives the same normalized audio.

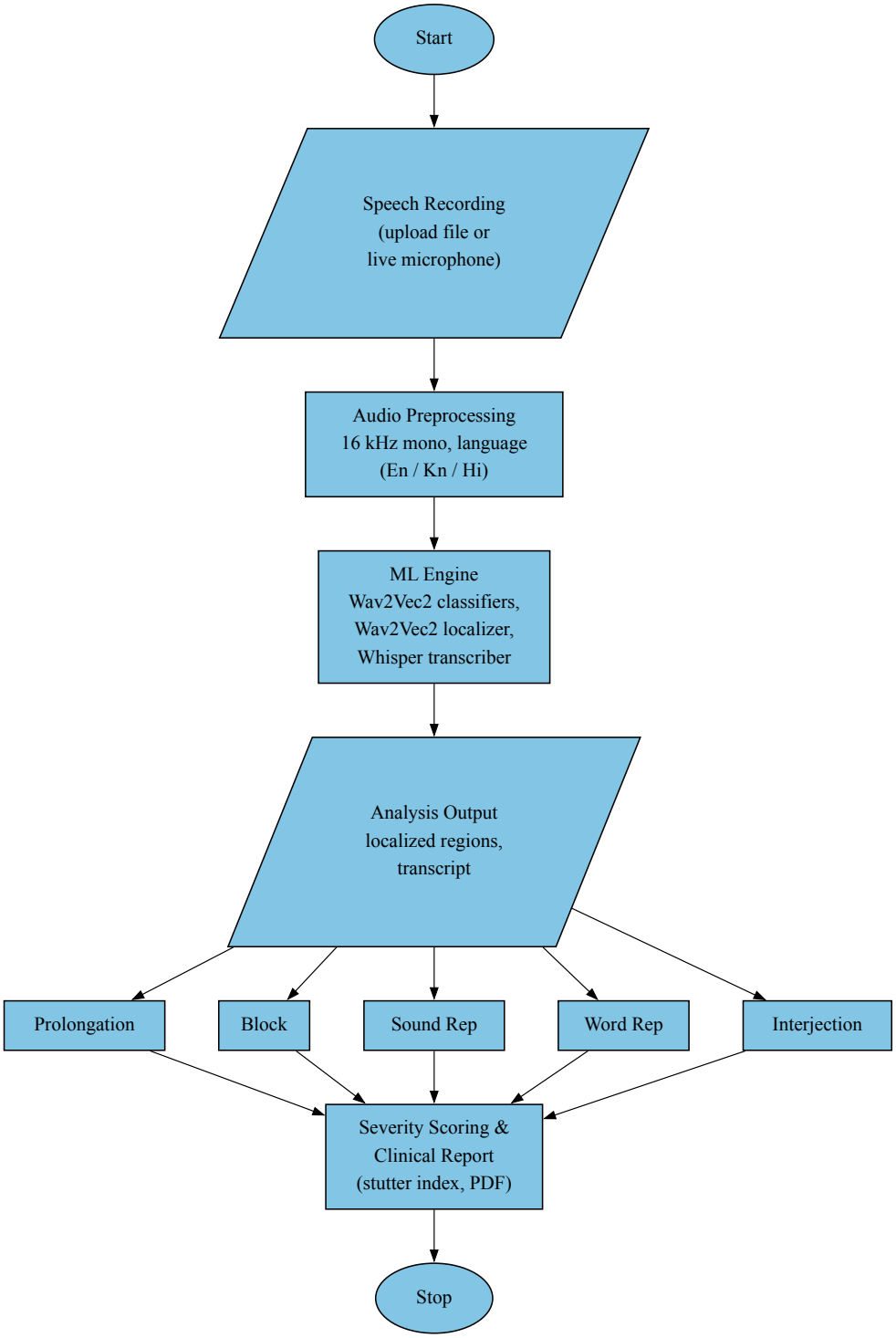


Figure 3.2: Flowchart

The flowchart in Figure 3.2 shows the full workflow, from input audio through preprocessing, the three model branches, and the final report. Initially, the Wav2Vec 2.0

model is used to extract high-level contextual speech features. To enhance performance, a hybrid model combining five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer is developed, where the classifiers capture dysfluency-specific acoustic patterns and the localizer identifies fine-grained temporal segments. The outputs from both pipelines are fused through CTC-based alignment and passed through a final aggregation layer that categorizes the audio samples as containing blocks, prolongations, sound repetitions, word repetitions, or interjections, along with the precise localized regions.

The system also defines a structured data flow for training. Training uses three datasets: Project Boli (sourced via Git repositories), SEP-28K (approximately 28,000 audio clips), and UCLASS (both obtained via Kaggle). Each dataset goes through its own normalization functions.

The processed datasets merge into a unified format: a combined_labels.csv file with multi-label binary annotations, plus individual interval CSV files for each audio clip. The merged dataset is split into training, validation, and testing subsets with an 80:10:10 split, with the training subset further re-split 80/20 during model training. The subsets are organized with symbolic links for efficient access, and preprocessed audio files are cached to speed up training.

3.4 USE CASE DIAGRAM AND DESCRIPTIONS

Figure 3.3 shows the use case diagram in UML: the actor stands outside the system boundary, and lines connect it to the use cases it can start. The only actor is the User, since clinicians and people assessing their own speech follow the same path.

Upload / Record Speech is the entry point. Audio can come from the microphone or a saved file, and the interface checks the file before sending it to the backend.

Select Language sets the language used for transcription and alignment; English, Kannada, and Hindi are supported.

Analyze Speech is the central use case. Full mode runs preprocessing, classification, transcription, localization, and CTC-based alignment. Classification-only mode runs the five classifiers and skips transcription and localization. Full mode returns a confidence score for each dysfluency type, a summary of the primary dysfluency, and the time regions where events were found.

View Analysis Results shows the per-class scores, a waveform with color-coded dysfluency regions, a spectrogram, and a timestamped transcript.

Download Clinical Report writes the stored result to a PDF without rerunning the models. Review Assessment History reopens earlier sessions from IndexedDB and localStorage, so recordings can be compared over time. Theme switching and the standardized reading passages support the same workflow.

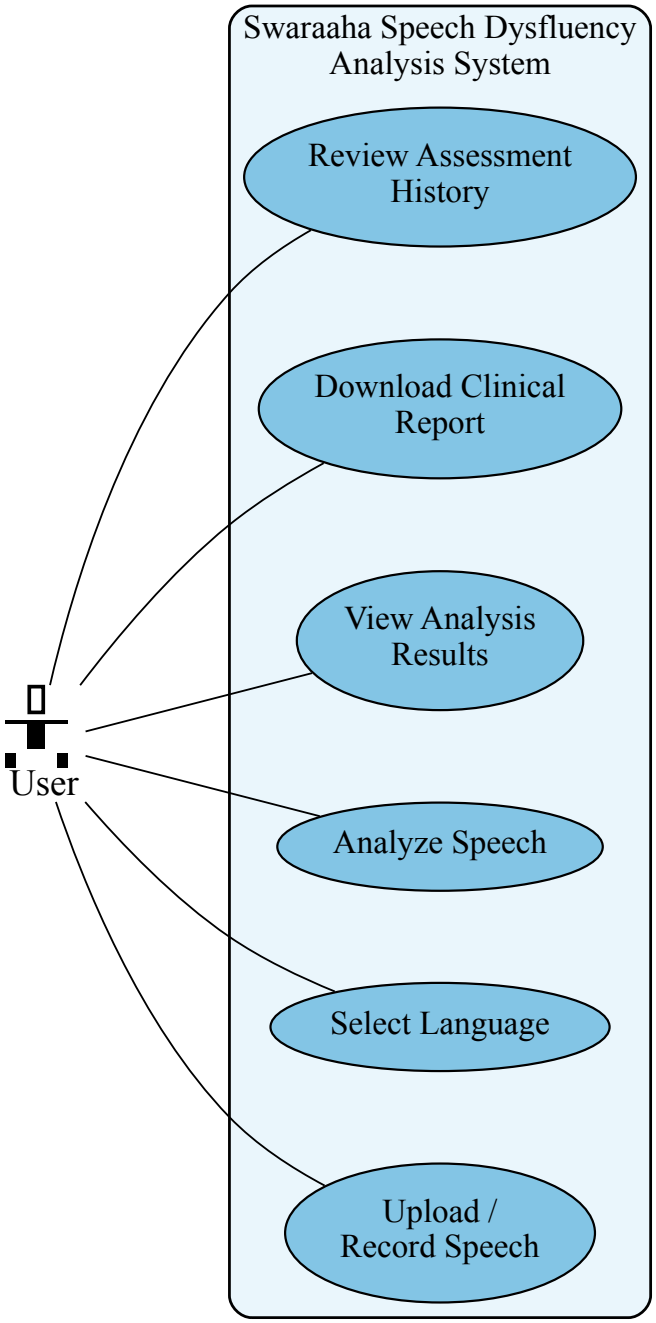


Figure 3.3: Use Case Diagram

Figure 3.3 groups the six use cases around the single actor. Upload / Record Speech and Select Language supply the audio and the language setting. Analyze Speech uses both and produces the classification scores, localized regions, and transcript. View Analysis Results, Download Clinical Report, and Review Assessment History then read the stored result. A report or a history entry cannot exist before an analysis has run. Because the

results page, the report, and the history all read the same payload, they show the same outcome for a given recording.

3.5 WORKFLOW DIAGRAM

A workflow diagram is a structured visual representation of a sequence of operations or tasks carried out to complete a specific process, typically used to analyse, design, or manage complex systems. It employs standardized symbols such as rectangles to denote actions, diamonds for decision points, and arrows to indicate the direction of flow, thereby offering clarity and insight into the procedural steps involved. In the context of deep learning and stutter detection, the workflow diagram illustrated here systematically maps out the process for developing a stutter classification and localization system.

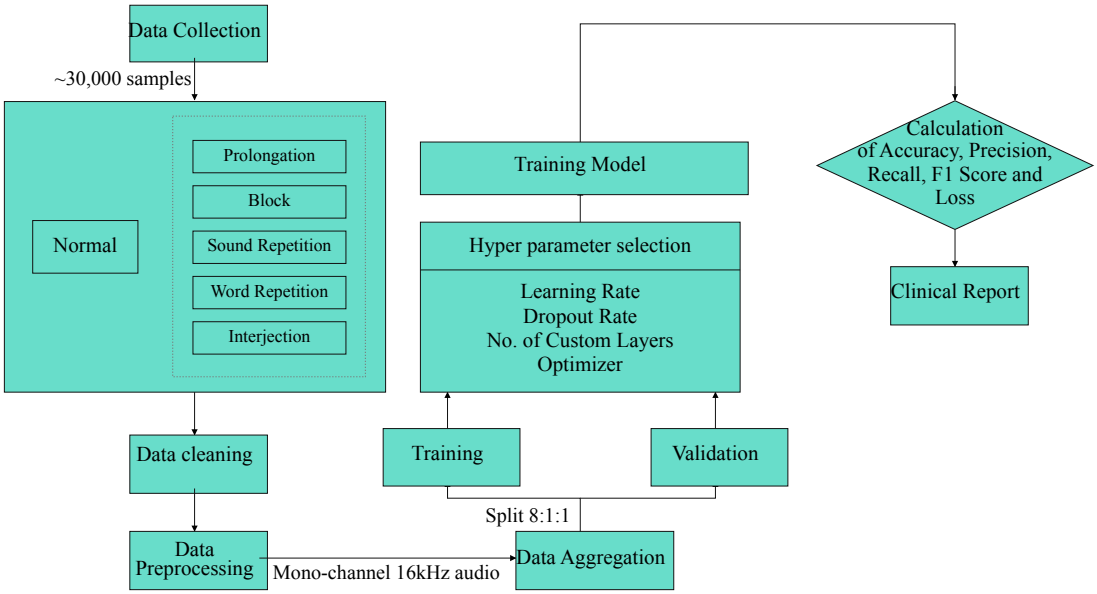


Figure 3.4: Workflow Diagram

The workflow of the proposed deep learning pipeline for stutter detection and localization is shown in Figure 3.4. The process begins with the collection of speech audio samples from three public stuttering datasets, Project Boli, SEP-28K, and UCLASS, all converted to 16 kHz mono and standardized through DC offset removal, peak normalization, silence trimming, and fixed-length padding. Data augmentation techniques such as random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling are applied to the waveforms, along with time and frequency masking on the spectrograms, expanding the effective dataset so the models generalize well across varied speaking patterns. The processed audio is then split into training, validation, and testing sets using an 80:10:10 ratio, with the training subset further re-split 80/20 during training. Model development begins with fine-tuning a pre-trained Wav2Vec 2.0 backbone, followed by

a hybrid architecture that combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer to capture both global dysfluency patterns and fine-grained temporal segments. Hyperparameters including learning rate, backbone freeze duration, optimizer, and the number of trainable layers are tuned to enhance performance, and the trained network is periodically validated to ensure stable learning and prevent overfitting. Finally, evaluation metrics such as precision, recall, F1-score, AUROC, and mean IoU are computed, and the best-performing model is selected for reliable automated assessment of speech dysfluencies.

3.6 SEQUENCE DIAGRAM

The sequence diagram shows how the components interact during execution. The user works through the graphical user interface (GUI), which sends a request to the backend API endpoint (/api/analyze).

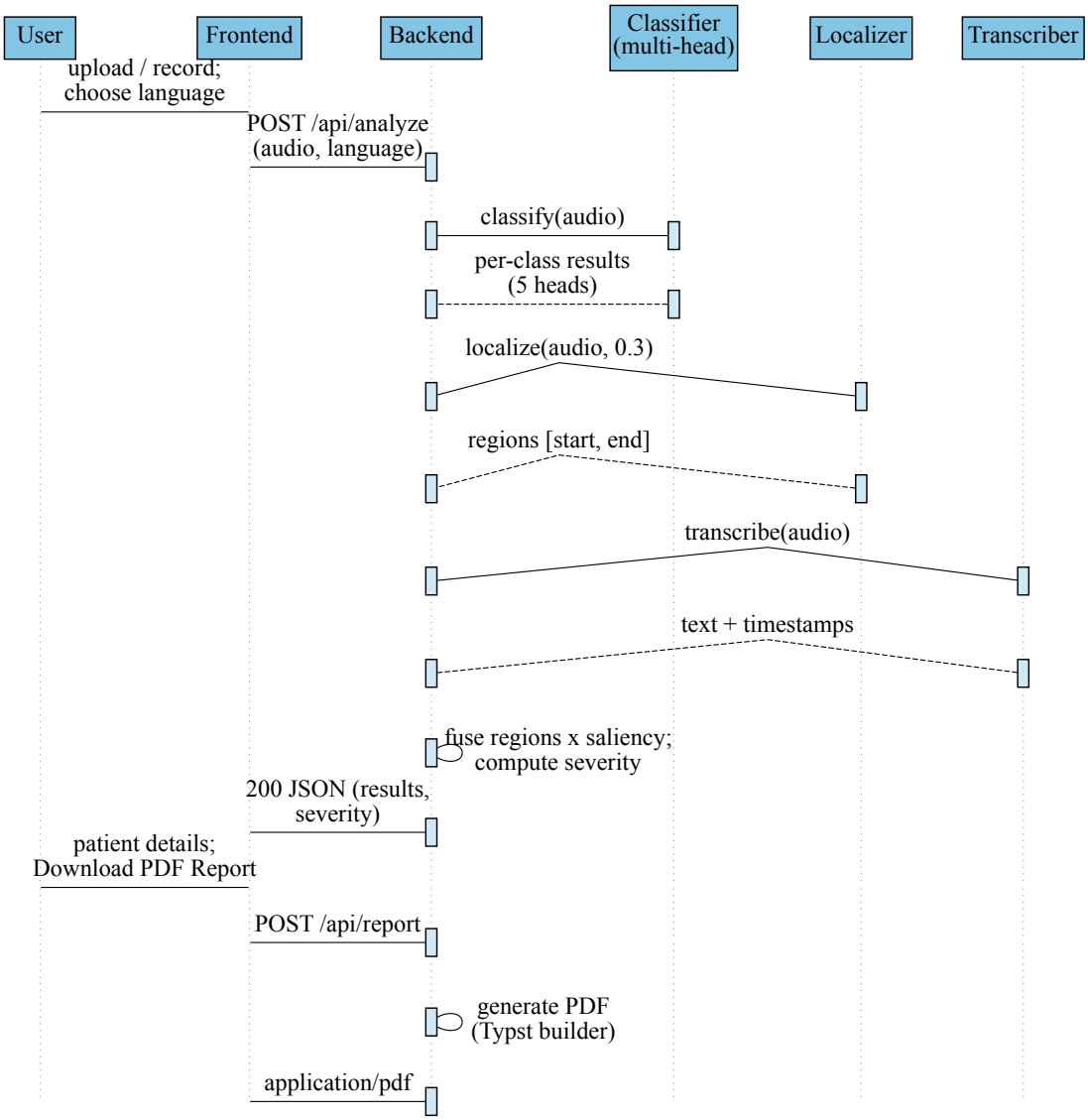


Figure 3.5: Sequence Diagram

The backend runs the request through a chain of services: preprocessing, classification, transcription, localization, and alignment. Each service does its part and passes its output to the next stage. When processing completes, the results go back to the frontend, where the user sees them. The system also stores the results for report generation and history management. This service sequence gives each component a clear responsibility and makes failures easier to identify. It also allows the stored analysis to be reused when the user opens a previous result, because the report layer receives the same structured payload that was produced during the original request.

The request lifecycle begins when the user confirms an analysis, at which point the frontend packages the audio, the selected mode, and any optional parameters into a single request. The backend validates the payload before any model is invoked, rejecting unsupported formats or empty inputs early so that expensive inference is never wasted on malformed data. Preprocessing normalizes the waveform and produces the log-mel spectrograms and frame-level features that the downstream stages expect, acting as the single point where audio conventions are enforced. Classification then assigns probability scores to each dysfluency type, and transcription produces a timestamped transcript that the alignment stage later couples to the detected events. Localization runs in parallel with transcription where possible, since the two stages share the same preprocessed audio and do not depend on each other's output, which keeps the overall latency low. The alignment stage fuses word timings with the localized regions so that each dysfluency can be reported in the context of the spoken words around it. If any single service fails, the backend returns a structured error instead of a partial result, and the frontend surfaces the failure without discarding the user's audio, allowing the request to be retried. On success, the fused payload is stored locally so that reports and the session history can be reconstructed later without re-running the models, which also keeps repeated report generation fast and independent of the analysis services. This ordering also supports responsive use of the application. Independent operations can run in parallel after preprocessing, while the final report waits only for the outputs required for alignment and presentation. The stored result preserves the predictions, timestamps, transcript, and derived annotations together, so the web and desktop interfaces can display the same analysis without applying different interpretation rules.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 INTRODUCTION

The system implementation process is a multifaceted endeavor focused on developing a robust software architecture capable of effectively managing diverse user interactions, data processing tasks, and model inference operations. By employing meticulous design and coding practices, the implementation ensures that the application is scalable, reliable, and secure, capable of handling varying workloads and protecting sensitive data. Iterative testing and debugging procedures are integral components of the implementation process, allowing for the identification and resolution of any issues or bugs that may arise. This iterative approach ensures that the application delivers a seamless user experience, free from disruptions or errors. Additionally, post-deployment monitoring and maintenance mechanisms are established to continually assess system performance and address evolving requirements. This ongoing maintenance ensures that the system remains optimized and responsive to user needs, even as conditions change over time, thereby facilitating long-term success and user satisfaction.

4.2 ALGORITHM

The inference process follows a fixed sequence of steps that turns raw speech input into a complete, annotated analysis result. Each step has a defined input and output, which limits the amount of hidden state passed between services. This structure is important for repeatable analysis because the same conversion, normalization, model execution, and reporting stages are used for both recorded and uploaded speech.

- Step 1, Input Acquisition: the system acquires speech input either through real-time recording or by uploading an audio file.
- Step 2, Audio Conversion: the input audio is converted into a 16 kHz mono WAV format using FFmpeg to maintain consistency across all inputs.
- Step 3, Preprocessing: the audio is cleaned by removing the DC offset, applying peak normalization to a target amplitude, and trimming silent segments.
- Step 4, Length Normalization: the processed audio is adjusted to a fixed length of 48,000 samples (three seconds at 16 kHz) by padding or truncating as required.

- Step 5, Parallel Processing: the system processes the audio simultaneously through three independent branches:
  1. Classification: Wav2Vec 2.0 embeddings are generated and passed through five binary classifiers, one per dysfluency type, and the outputs are aggregated into a multi-label result with a probability score for each dysfluency type.
  2. Transcription: the Whisper model generates a timestamped transcript of the speech in English, Kannada, or Hindi.
  3. Localization: the Wav2Vec2 frame-level localizer identifies the frame-level regions within the audio where dysfluencies occur.
- Step 6, Alignment: the detected dysfluency regions are mapped to the corresponding words or syllables in the transcript using CTC-based time alignment.
- Step 7, Combination and Severity: the combiner labels each localized region with per-class saliency scores from the classifier and fuses the classification and localization views, and the severity module computes a stutter index from the ratio of dysfluent speech duration to total speech duration.
- Step 8, Visualization and Report Generation: the system displays the results through waveform overlays, spectrograms, transcripts, and confidence scores, and generates a detailed clinical-style report.

4.3 IMPLEMENTATION REQUIREMENTS

This section presents the implementation requirements for detecting and localizing stuttering with a hybrid deep learning model that combines Wav2Vec 2.0 feature extraction, five independent Wav2Vec 2.0 binary classifiers, a Wav2Vec2 frame-level localizer, and Whisper automatic speech recognition. The main objective is to build a reliable, efficient system that identifies speech dysfluencies and localizes them within the audio. The hybrid model uses the strengths of both architectures: Wav2Vec 2.0 supplies self-supervised, contextual speech representations that capture fine-grained phonetic and prosodic patterns, while Whisper provides sequence-level transcription that helps map localized dysfluency regions to specific words and syllables.

The implementation uses the Python ecosystem, which provides an interactive environment for model development, experimentation, and analysis. Python serves as the primary language for backend development because of its versatility and its support for a wide range of scientific and deep learning libraries such as PyTorch, the Hugging Face Transformers library, and NumPy, which help with model construction, training, and evalua-

tion. For the frontend, React 19 and PySide6 build a user-friendly web interface and desktop application where users can record or upload speech audio and view classification and localization results. All trained models are managed by a centralized model registry configured through a registry.json file that maps each task to its checkpoint and per-class thresholds, so checkpoints can be updated or replaced without changing application code. The Python ecosystem also facilitates matrix manipulation, data visualization, algorithm implementation, and seamless integration with other programming environments, making it an ideal platform for developing and testing the hybrid model for stuttering classification and localization.

4.3.1 Train the Model

Training is a key step in developing the system for stutter classification and localization. The speech audio dataset is first collected from Project Boli, SEP-28K, and UCLASS, then preprocessed through conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding, with data augmentation to improve audio quality and reduce overfitting. The Wav2Vec 2.0 model is then trained to extract high-level contextual speech features, using its self-supervised pretraining to learn robust representations. To improve classification accuracy further, the hybrid model combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer. The localizer captures fine-grained temporal features at the frame level, complementing the broader segment-level extraction of the classifiers. CTC-based time alignment then fuses the outputs of both models into one feature representation. Training uses the AdamW optimizer to adjust learning rates adaptively and keep convergence stable. It also uses mini-batches of audio, and backpropagation updates the weights from the computed loss. Validation runs periodically to monitor accuracy and prevent overfitting. After training, the optimized parameters and weights are saved, so the models can predict on new speech audio without retraining.

4.3.2 Test the Model

Testing happens after training to evaluate how well the proposed model detects stutter events in unseen speech audio. When a user records or uploads a sample through the web interface or desktop application, it goes through the same preprocessing used in training: conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding. This keeps the input uniform for accurate analysis. The trained Wav2Vec 2.0 model then extracts high-level contextual speech features. For higher accuracy, the audio also runs through the hybrid model that combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer, which

captures both segment-level dysfluency patterns and fine temporal detail. CTC-based time alignment fuses the features from both models into one representation. The final aggregation layer then produces a classification result that names the dysfluency types present (blocks, prolongations, sound repetitions, word repetitions, or interjections) and the precise regions involved. The interface shows the outcome with waveform overlays, spectrograms, a timestamped transcript, and confidence scores. The model's predictions are also compared with ground truth labels to measure accuracy and generalization. This phase confirms that the hybrid model performs well on new and unseen samples, which points to reliable stutter classification and localization in real clinical use.

4.3.3 Model Evaluation

Model evaluation is essential for assessing the overall performance and reliability of the proposed system. Once the models are trained and tested, several quantitative metrics measure their classification performance:

- Accuracy: measures the proportion of correctly predicted samples.
- Precision: the share of predicted positive instances that are actually positive.
- Recall (Sensitivity): measures how effectively the model identifies all relevant positive samples.
- F1-Score: provides a harmonic mean of precision and recall to balance both metrics.

The five Wav2Vec 2.0 binary classifiers, one per dysfluency type, are further measured with AUROC (Area Under the Receiver Operating Characteristic Curve), AUPRC (Area Under the Precision-Recall Curve), and specificity, summarized per class and as a macro average across the five dysfluency classes. The Wav2Vec2 frame-level localizer is measured with frame-level precision, recall, and F1, detection accuracy, mean Intersection over Union (IoU) between predicted and ground-truth regions, and the false alarm rate per minute.

A confusion matrix for each of the five binary classifiers shows the distribution of predictions, including true positives, false positives, true negatives, and false negatives. This evaluation shows the strengths and weaknesses of each class; interjections, for example, are detected most reliably, while blocks are the hardest.

All reported results use the held-out test set of 3,715 clips, which is never touched during training or threshold selection, together with a cross-corpus Boli set that checks how well the models transfer to unseen speakers.

4.4 USER INTERFACE

The web-based interface connects the user to the proposed stutter detection and localization system. Users record speech or upload audio files for analysis. Once audio is provided, the backend system carries out the preprocessing steps, followed by feature extraction, classification, transcription, and localization using the trained models. Initially, the Wav2Vec 2.0 model is used to extract high-level contextual speech features, and to achieve higher accuracy, the hybrid pipeline combining five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer is employed. Within a few seconds, the system presents the predicted result on the screen for each of the five dysfluency types, along with confidence scores, the localized regions, and the timestamped transcript. The tool is designed for clinicians, speech-language pathologists, researchers, and individuals who stutter.

The system provides two interfaces to the same analysis pipeline: a web application and a desktop application. The web frontend is built with React 19, TypeScript, and Tailwind CSS, with a sidebar layout covering the input page, results page, history, and reading passages. The input page supports audio file upload (WAV, MP3, FLAC, M4A, OGG, WMA) and direct microphone recording through the MediaRecorder API, with client-side validation before the file is sent to the backend as multipart form data. The results page shows classification results with confidence scores, an interactive waveform with color-coded dysfluency region overlays, a timestamped transcript, and a button to generate and download a PDF report. The backend runs on FastAPI served through Uvicorn, exposing the /api/classify, /api/localize, /api/analyze, and /api/report endpoints. Assessment history stays in the browser: audio files go into IndexedDB for offline access and analysis metadata into localStorage.

The desktop application is built with PySide6 (Qt for Python) for a native cross-platform interface. It includes modules for audio recording and playback (sounddevice), file handling (soundfile), and model inference, calling the shared model package directly with no separate server, so it works fully offline. PDF report generation uses Typst with pypdfium2 for viewing.

Both interfaces share the same model package, preprocessing conventions, and result schema, so a recording analysed in the browser and the same recording analysed on the desktop produce identical predictions and reports. The web client keeps long-running requests responsive by streaming progress updates and disabling duplicate submissions while an analysis is in flight, while the desktop client runs inference in a worker thread so the window stays interactive during transcription and localization. Theme switching,

keyboard navigation, and consistent layout across window sizes are handled at the component level, which keeps the interface usable for clinicians who move between machines and screen sizes. Uploaded audio is validated on the client before it reaches the backend, and the results view keeps the waveform, transcript, and per-class confidence table in sync so that selecting a timestamp jumps to the corresponding part of the recording. Report generation is deliberately decoupled from analysis: the stored result payload is sufficient to rebuild the PDF, which means a report can be regenerated with a different layout or header without repeating the expensive model inference. This shared design also simplifies maintenance, because a change to the result schema or the report template propagates to both interfaces from a single source rather than being reimplemented twice.

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

Testing is a crucial phase in the development of the proposed system for stutter detection and localization, ensuring the overall accuracy, stability, and reliability of both the individual models and the hybrid pipeline. Roughly one percent of people worldwide are affected by stuttering, and diagnosis today still relies mostly on manual assessment by speech-language pathologists, which is subjective, time-consuming, and not available to everyone, making early and reliable detection essential. In this research, Wav2Vec 2.0-based models were initially utilized to classify speech dysfluencies from audio. To further enhance diagnostic precision, a hybrid framework was developed by combining five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer and Whisper automatic speech recognition, enabling classification, localization, and transcription in a single pipeline. The testing process involved validating every stage of the pipeline, including data preprocessing, model training, classification, localization, and web-based visualization, through unit, integration, functional, and system tests. Comprehensive testing confirmed that the proposed system satisfies all functional and non-functional requirements while maintaining reliable classification and localization. Performance evaluation using precision, recall, F1-score, AUROC, AUPRC, specificity, and confusion matrix analysis, along with evaluation on the held-out test set and a cross-corpus set, demonstrated the robustness and generalization ability of the models. The results are broken down by model family and dysfluency class, and the two model families, the Wav2Vec2-based models and the CNN-based models, are compared against each other and analyzed for their strengths and weaknesses.

5.2 TYPES OF TESTS

Four types of tests were performed: unit testing, integration testing, functional testing, and system testing. Together they verify the correctness of individual components, the interfaces between them, the complete feature set, and the behavior of the finished system.

5.2.1 Unit Testing

Unit testing isolates the smallest blocks of code, specific functions and classes, to verify they work as expected. The project uses pytest suites organised under model/tests, backend/tests, app/tests, and shared/tests.

Unit tests verify the preprocessing steps (sample-rate conversion, DC offset removal, peak normalization, silence trimming, and fixed-length padding), dataset normalization and the merged combined_labels.csv format, model registry checkpoint loading, the combiner and CTC alignment logic, and the computation of the evaluation metrics. On the desktop side, individual widgets such as the waveform view, results panel, and audio handler are tested in isolation. Ruff enforces consistent code style across the codebase.

5.2.2 Integration Testing

Integration testing ensures that software modules, when connected, work together seamlessly. The backend tests exercise the FastAPI endpoints (/api/analyze, /api/classify, /api/localize, and /api/report), which chain preprocessing, classification, transcription, localization, and fusion into a single analysis response. The classification, fusion, localizer, severity, and report generator services are tested as combined units, and the model registry is checked against the classifier and localization pipelines. The desktop application calls the shared model package directly, and its model runner is integration-tested against the same service layer.

5.2.3 Functional Testing

Functional testing validates the complete set of features against the specifications under typical and edge cases. The workflow of recording or uploading audio, running the analysis, and viewing the results page with classification scores, waveform overlays, and the timestamped transcript is exercised end to end. Report generation and download, severity mapping, multilingual transcription for English, Kannada, and Hindi, and history storage through IndexedDB and localStorage are all verified against the requirements defined in Chapter 2.

5.2.4 System Testing

System testing subjects the whole software product to end-to-end scenarios in conditions that resemble the user environment. The React web application was exercised in supported browsers and the PySide6 desktop application in a standalone offline setup with no separate server, both against the same machine learning backend. Supported audio formats (WAV, MP3, OGG, and M4A) were validated across the upload, drag-and-drop, and microphone capture paths, corrupted files were rejected with clear error messages, and analysis turnaround was confirmed to stay within a few seconds per clip as required by the performance requirements in Chapter 2. The combined system-level suite confirmed that the application meets its functional and non-functional requirements.

5.3 RESULT AND ANALYSIS

This section presents the evaluation of the trained models: how the Wav2Vec2-based and CNN-based model families were trained and evaluated, the hyper parameter tuning applied, and a comparative performance analysis of all architectures.

5.3.1 Model Evaluation of the Wav2Vec2 Classifiers

The model registry defines two Wav2Vec2-based classification configurations: the default single configuration of five independent Wav2Vec2 binary classifiers, and a multitask variant that shares a single backbone and five classification heads. This section first analyzes the multitask variant in detail and then reports the shipped single configuration; the confusion matrices at the end of the section come from the shipped single configuration.

The multitask variant uses a single Wav2Vec2-base backbone with five shared binary heads. Training used Focal Loss (gamma=2.0), backbone freezing for 3 epochs, AdamW (LR 3×10^{-5}), a 500-step warm-up, and early stopping with a patience of 5. Results on 3,715 held-out test samples at the default threshold of 0.5 are in Table 5.1.

Class	**Precision**	**Recall**	**F1**	**AUROC**	**AUPRC**	**Support**
Prolongation	0.566	0.440	0.495	0.845	0.491	400
Block	0.584	0.264	0.364	0.768	0.444	553
Sound Repetition	0.705	0.502	0.586	0.878	0.624	494
Word Repetition	0.621	0.399	0.486	0.855	0.550	366
Interjection	0.810	0.746	0.776	0.935	0.848	834
Macro Average	–	–	**0.542**	–	–	**3,715**

Table 5.1: Multitask Wav2Vec2 classifier results

The per-class support counts cover positive samples; the macro row reports the total of 3,715 test clips because a clip may contain several stutter types at once.

The macro F1 at the default threshold is 0.542. Interjection performs best (F1=0.776, AUROC=0.935), which makes sense because filler words like “um” and “uh” have distinct acoustic patterns. Block is the hardest class (F1=0.364); silent pauses and hesitations are hard to distinguish from normal speech pauses.

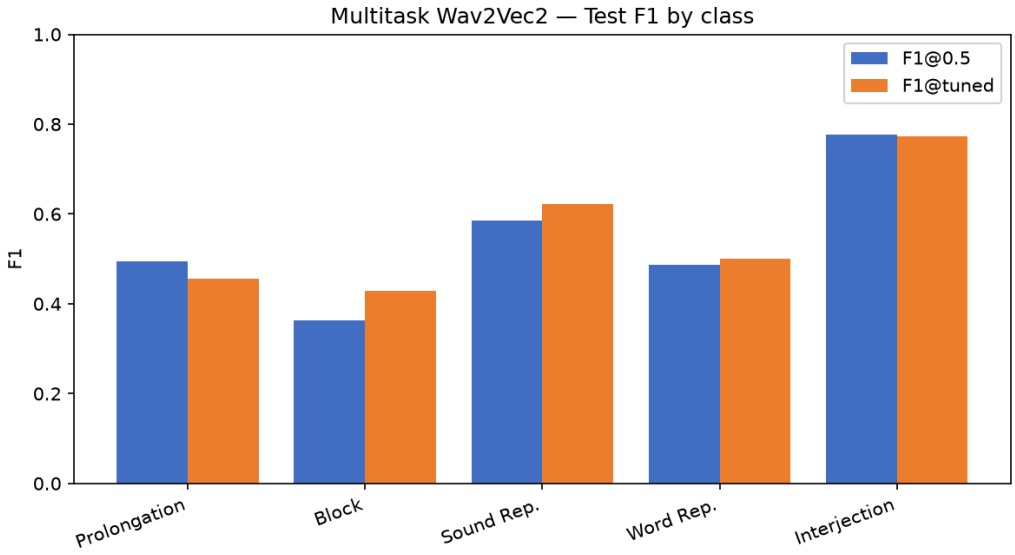


Figure 5.1: Per-class F1 of the multitask Wav2Vec2 classifier

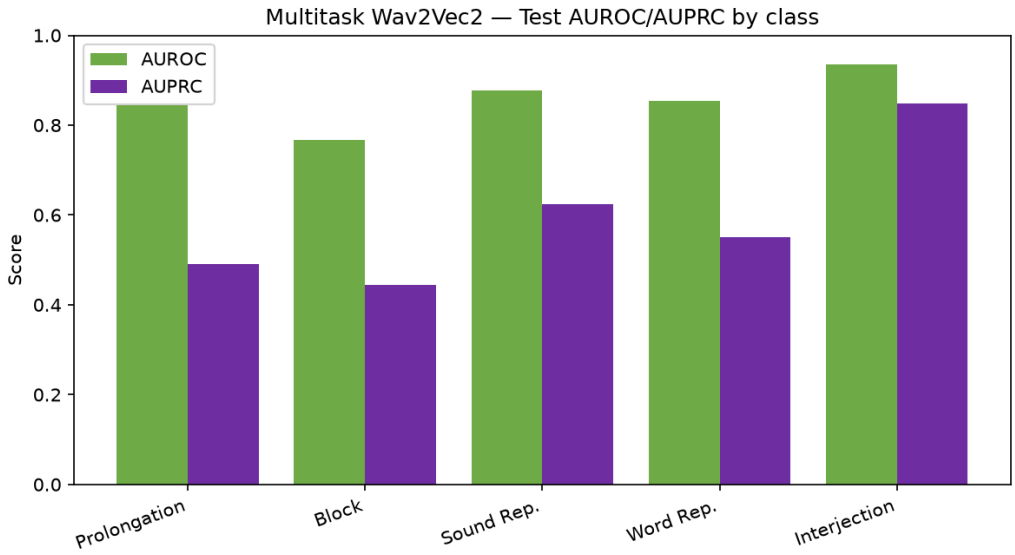


Figure 5.2: Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier

The per-class F1 scores are plotted in Figure 5.1, and the corresponding AUROC and AUPRC values are plotted in Figure 5.2. Both figures mirror the table above: interjection leads on every measure, while block trails on F1 even though its AUROC and AUPRC stay reasonably high.

The shipped single configuration of five independent binary classifiers scores slightly higher in distribution, with a test F1 of 0.573 at threshold 0.5 (0.570 with tuned thresholds).

The confusion matrices for the five dysfluency types are shown in Figure 5.3 and visualize the distribution of predictions of the shipped single configuration of five independent binary classifiers.

Before examining the matrices, it is useful to recall how the aggregate scores were obtained. Each binary classifier was trained with a class-balanced objective, and the operating threshold was selected on the validation split rather than the test split, so the matrices show the behaviour of the shipped configuration at its default threshold rather than a threshold tuned to inflate the reported numbers. Reading them row by row reveals how often each event type is correctly recovered, while reading them column by column shows which fluent or other-type segments are most likely to be mistaken for it. This distinction matters in practice, because a clinician cares both about missed events and about false alarms that interrupt a session.

The matrices are arranged so that the true label is on one axis and the predicted label on the other, with darker cells indicating larger counts. A classifier that simply predicted the majority class would produce a single dark row, which none of the five matrices does, confirming that each head learned a genuine decision boundary rather than collapsing to the dominant label. The remaining off-diagonal mass is concentrated in specific cells, which points to concrete confusions that later sections address through threshold tuning and post-processing.

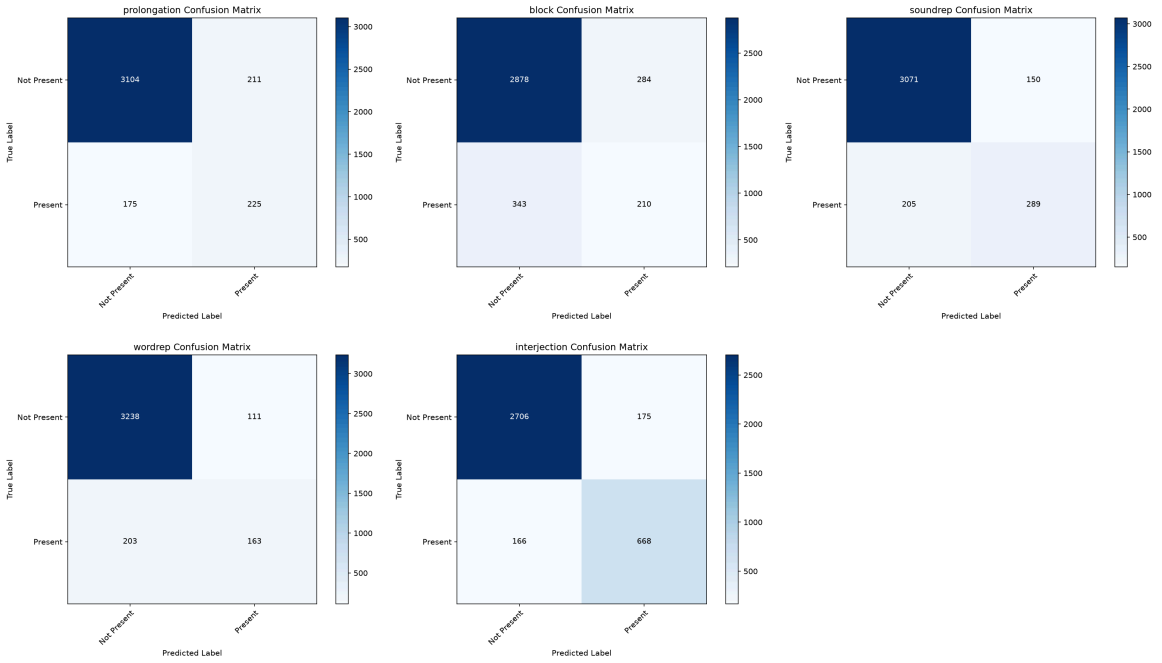


Figure 5.3: Confusion matrices for the five binary classifiers

Each confusion matrix contrasts the true labels against the predicted labels for one binary classifier, so the diagonal cells count correct decisions and the off-diagonal cells expose the direction of error. A strong diagonal with a light upper-right corner means the classifier rarely over-triggers, while a heavy lower-left corner indicates missed events, and the balance between the two is what separates a conservative detector from an over-sensitive one. Interjection shows the cleanest diagonal, consistent with its high F1 and AUROC,

because its acoustic signature, typically a short vocalized filler, is spectrally distinct from fluent speech. Block and word repetition show more spread, since prolonged silences and repeated words can resemble pauses and normal hesitation, which the classifiers then confuse at the boundary. Reading the matrices alongside the per-class table makes the trade-off explicit: a classifier tuned for high recall shifts mass into the false-positive corner, while one tuned for precision leaves more events undetected. This is precisely why the shipped system keeps separate binary classifiers instead of a single multi-class head, since each event type can be calibrated to its own operating point without disturbing the others.

The Wav2Vec2-based model family also provides the frame-level localizer, evaluated on the same 3,715 test samples at a threshold of 0.5; the shipped registry inference threshold for this localizer is 0.3. Results are in Table 5.2.

Metric	Value	Metric	Value
Frame Precision	0.805	Detection Accuracy	0.586
Frame Recall	0.738	Mean IoU	0.779
Frame F1	0.770	False Alarms/min	4.28
Frame Specificity	0.847	Predicted Events	1,846

Table 5.2: Wav2Vec2 localizer evaluation results

The localizer detects most dysfluency events with good accuracy: frame precision is 0.805 and recall is 0.738, so most true events are found and most predictions are correct. The mean IoU of 0.779 means the regions it finds overlap well with ground truth. Of 995 true events it predicted 1,846, of which 1,263 were false alarms (4.28 per minute).

The gap between frame-level and event-level behaviour is worth emphasizing. A frame F1 of 0.770 means that most individual frames are labelled correctly, but the event count shows that the localizer predicts nearly twice as many events as are actually present, so a clinician using the raw output would see clusters of short, spurious detections around genuine events. The 4.28 false alarms per minute figure is the practical consequence of that over-prediction, and it is the main reason the shipped registry applies a higher inference threshold of 0.3 rather than the 0.5 used for reporting. Raising the threshold trades recall for precision by removing low-confidence detections, which reduces the false-alarm rate at the cost of occasionally dropping a weak but real event. The choice of threshold is therefore a clinical decision as much as a technical one, and the system exposes it as a configurable parameter so that a screening workflow can favour sensitivity while a detailed assessment can favour precision. The following subsection turns to how

the classifiers behind these detections were trained, since the localizer and the classifiers share the same pretrained backbone and the same preprocessing conventions.

These frame-level metrics are visualized in Figure 5.4.

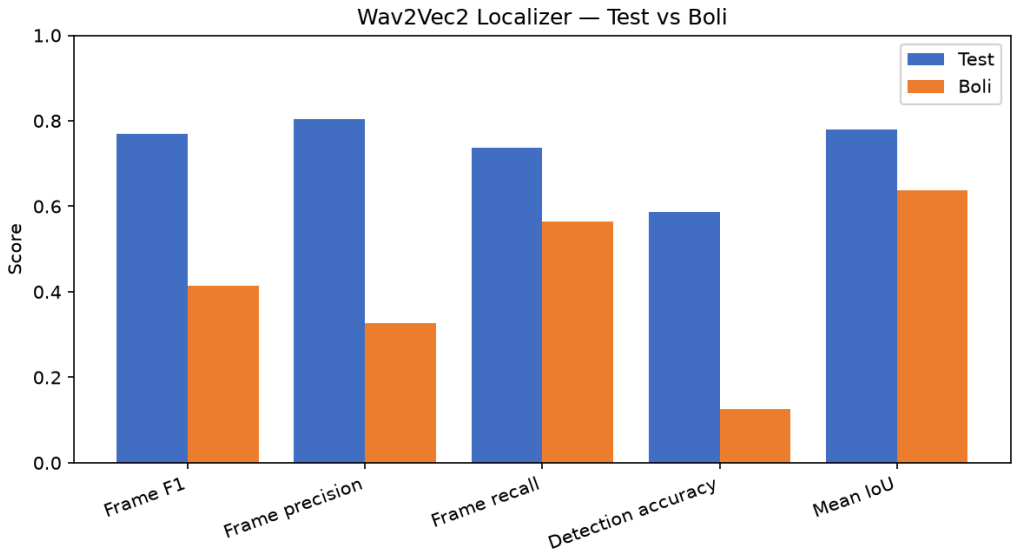


Figure 5.4: Frame-level metrics of the Wav2Vec2 localizer

5.3.2 Wav2Vec2 Classifier Training

The training phase prepares the models that the system relies on for classification and localization.

The system is trained and evaluated on three publicly available stuttering datasets: Project Boli, SEP-28K, and UCLASS. Project Boli is a multilingual Indian language dataset with stuttering data across Hindi, Kannada, Telugu, and other Indian languages, sourced from GitHub, and includes both read and spontaneous speech. The ingestion resolves only the English task codes of this dataset, so the Boli material used here is English speech; it is therefore a cross-corpus rather than a cross-lingual evaluation set. SEP-28K provides roughly 28,000 audio clips from stuttering support group podcasts, annotated for blocks, prolongations, sound repetitions, word repetitions, and interjections, and was obtained from Kaggle. UCLASS is an additional stuttering speech dataset from Kaggle that provides supplementary annotated clips. Each dataset is normalized into a unified format with a combined_labels.csv file of multi-label binary annotations and interval files per clip. The merged dataset is split 80:10:10 into training, validation, and testing subsets, and during training the training subset is re-split 80/20 by a stratified split into a fine-tuning set and a validation set used for early stopping and threshold selection.

Each classifier is trained in a two-stage scheme. The backbone stays frozen for the first three epochs so the classification head can stabilize, then it is unfrozen with a learning

rate scaled to 0.1 of the head learning rate. Focal Loss with a gamma value of 2 handles the class imbalance, and the AdamW optimizer runs at a learning rate of 3×10^{-5} with a weight decay of 0.01 and a 500-step warm-up. Early stopping with a patience of 5 epochs prevents overfitting, and the checkpoint with the highest macro F1-score on the validation set is saved as the final model. During training, data augmentation applies waveform-level transforms (random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling) and spectrogram-level transforms (time and frequency masking). Checkpoints are stored under names that encode their training hyperparameters through a fingerprint convention, with checkpoint-based resume, an audio preprocessing cache for fast re-runs, torch.compile for roughly twice the training speed on CUDA, and gradient accumulation for effective batch size scaling.

Several problems encountered during training were resolved. BCEWithLogitsLoss with a single logit per class created a zero-gradient equilibrium where the model collapsed to always predicting the majority class; switching to CrossEntropyLoss with two logits (present and not present) fixed this. Gradient clipping in the classifier loop silently killed learning by scaling the classifier head gradients down, so it was removed from the classification loop while the localizer loops retained it. Mixed-precision training with a GradScaler produced NaN gradients; using torch.autocast alone worked. Freezing the backbone for 3 epochs let the classifier head stabilize before fine-tuning, and longer freeze durations hurt performance.

5.3.3 Model Evaluation of the CNN-based Models

The CNN-based models form the second model family, built to run classifications with far fewer parameters and to compare generalization against the pre-trained Wav2Vec2 baseline.

Four CNN-based multitask classification architectures were evaluated: CNN Pool, CNN Single, CNN-LSTM, and CNN-Transformer, ranging from 0.27M to 0.54M parameters. On the held-out test set they score markedly lower than the Wav2Vec2 classifiers, with tuned F1 values between 0.249 and 0.271. On the cross-corpus Boli set, however, the ranking flips: CNN Single reaches 0.476 and CNN-LSTM 0.508, the best cross-corpus result of all seven architectures, suggesting that the lighter models generalize better to unseen speakers.

The CNN spectrogram localizer was evaluated on the same 3,715 test samples at a threshold of 0.5. It is far more permissive than the Wav2Vec2 localizer: frame recall is high (0.873) but precision is much lower (0.527), and it predicted 4,483 events against 995 true events, with 4,329 false alarms (23.49 per minute). Its detection accuracy (0.155) and

specificity (0.330) are correspondingly low. The heavier Wav2Vec2 localizer is therefore shipped as the default; the registry inference threshold is 0.3 for both localizers.

5.3.4 CNN-based Model Training

The CNN-based models are trained on the same unified dataset and with the same two-stage scheme used for the Wav2Vec2 classifiers, but they consume mel-spectrograms instead of raw audio embeddings. CNN Pool applies global pooling over the convolutional feature maps, CNN Single processes a single spectrogram path, CNN-LSTM adds a recurrent layer to capture temporal context, and CNN-Transformer uses a transformer encoder over the spectrogram features. Spectrogram-level augmentation, namely time and frequency masking, is applied during training alongside the waveform-level transforms. Because these networks have only 0.27M to 0.54M parameters compared with roughly 94M to 97M for the Wav2Vec2 family, they train quickly and were used to study the trade-off between parameter count and cross-corpus generalization.

5.3.5 Hyper Parameter Tuning

A threshold sweep on the validation set tested thresholds from 0.1 to 0.9 in steps of 0.05 for each class. The threshold that maximized F1 for each class was stored in the model registry under the multitask configuration. The results are in Table 5.3.

Class	**Default (0.5) F1**	**Optimal Threshold**	**Tuned F1**	**Improvement**
Prolongation	0.495	0.55	0.457	−7.8%
Block	0.364	0.35	0.429	+17.8%
Sound Repetition	0.586	0.40	0.621	+6.0%
Word Repetition	0.486	0.35	0.499	+2.8%
Interjection	0.776	0.45	0.773	−0.5%
Macro Average	**0.542**	–	**0.556**	**+2.6%**

Table 5.3: Threshold optimization results

Threshold tuning raised the macro F1 from 0.542 to 0.556. Block improved the most (0.364 to 0.429) because the default threshold of 0.5 is too high for a class with strong imbalance. For prolongation and interjection the tuned threshold lowers F1 slightly on the test set, but per-class tuning on the validation set still yields a net gain of +2.6% overall.

Additional hyper parameters were tuned during development. The backbone freeze duration was compared at 3 and 20 epochs; freezing for 3 epochs (MT frz3) clearly beats 20 epochs (MT frz20) on the test set, showing that early unfreezing matters for fine-tuning. The learning rate scaling factor of 0.1 between the head and the unfrozen backbone, a

500-step warm-up, a weight decay of 0.01, and gradient accumulation for effective batch size scaling together gave the most stable training runs.

5.3.6 Performance Analysis of the Models

Seven architectures were compared in total, ranging from the shipped five independent Wav2Vec2 classifiers to various CNN-based multitask models. Test set and cross-corpus Boli results are in Table 5.4.

Architecture	Type	Parame-ters	Test F1 (0.5)	Test F1 (Tuned)	Boli F1 (Tuned)
5x Wav2Vec2	Classifier	94.6M	0.573	0.570	0.292
MT Wav2Vec2 (frz3)	Multitask	97.3M	0.542	0.556	0.238
MT Wav2Vec2 (frz20)	Multitask	97.3M	0.141	0.378	0.214
CNN Pool	Multitask	0.34M	0.149	0.256	0.332
CNN Single	Multitask	0.27M	0.212	0.262	0.476
CNN-LSTM	Multitask	0.46M	0.242	0.249	0.508
CNN-Transformer	Multitask	0.54M	0.269	0.271	0.345

Table 5.4: Comparative study of seven architecture variants

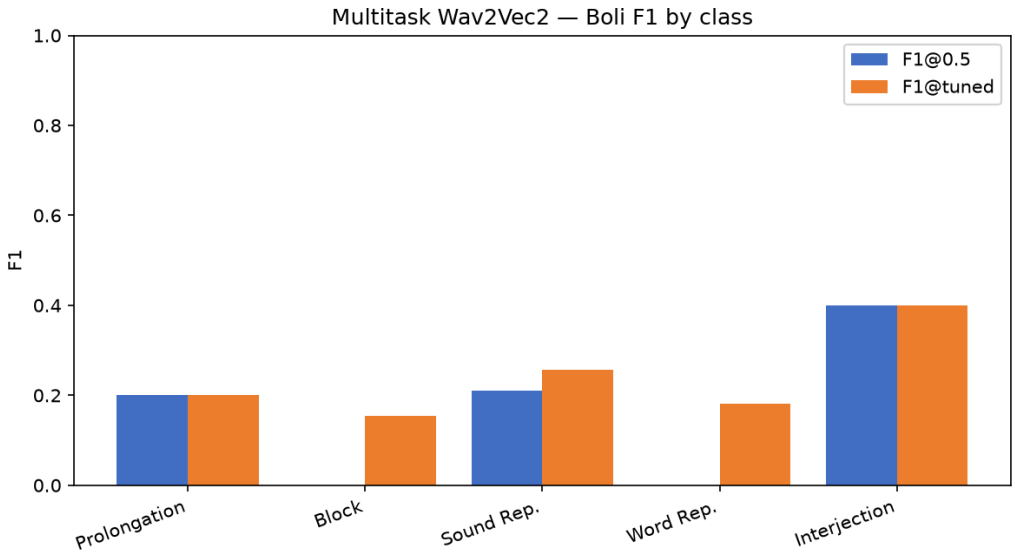


Figure 5.5: Per-class tuned F1 on the Boli set

The shipped five-classifier Wav2Vec2 configuration scores highest in distribution (tuned F1 0.570), narrowly ahead of the multitask variant (0.556); both benefit from large pre-trained speech representations. The CNN models have far fewer parameters but score lower on the test set. On the Boli cross-corpus set the ranking flips: CNN-LSTM reaches 0.508, the best cross-corpus result, followed by CNN Single (0.476). Lighter models

seem to generalize better to unseen speakers, while heavy pre-trained models do better on familiar data.

The per-class tuned F1 scores on the Boli set are shown in Figure 5.5, where the lighter CNN variants lead the comparison. This result reinforces the need to report both ordinary test performance and cross-corpus performance. The former measures how well the models recognize patterns similar to those seen during development, whereas the latter exposes sensitivity to changes in speaker, language, recording style, and data source.

It was concluded that the Wav2Vec2 family should be shipped for in-distribution accuracy, with the CNN-LSTM variant available as a lighter alternative that transfers best to unseen speakers and languages.

This conclusion reflects a deliberate trade-off rather than a claim that one architecture dominates on every axis. The Wav2Vec2 classifiers benefit from representations learned on large amounts of unlabeled speech, which is why they lead on the test set where the speakers and recording conditions resemble the training distribution. On the Boli corpus, where speakers and languages differ from the podcast-style training data, the advantage narrows and the smaller CNN models transfer better, suggesting that heavy pretrained backbones overfit to the acoustic characteristics of their training corpora. The practical implication is that the system reports both numbers instead of a single headline score, and that deployment should state which regime is expected. For a clinic working with speakers and equipment similar to the training data, the Wav2Vec2 configuration is the better default; for a deployment that must handle unfamiliar speakers or languages without retraining, the CNN-LSTM variant is the safer choice and runs on far more modest hardware.

The comparison also clarifies where further work would pay off: the gap between in-distribution and cross-corpus performance is larger than the gap between architectures, so data coverage and speaker diversity matter more than adding parameters. These findings also guide how the system should be evaluated after deployment. Performance should be monitored separately for familiar and unfamiliar recording conditions, and results should be reviewed by dysfluency class rather than reduced to one overall score. That reporting practice makes changes in data coverage, thresholds, or model architecture visible and helps prevent an improvement for one class from masking a regression in another.

CHAPTER 6

SCREENSHOTS

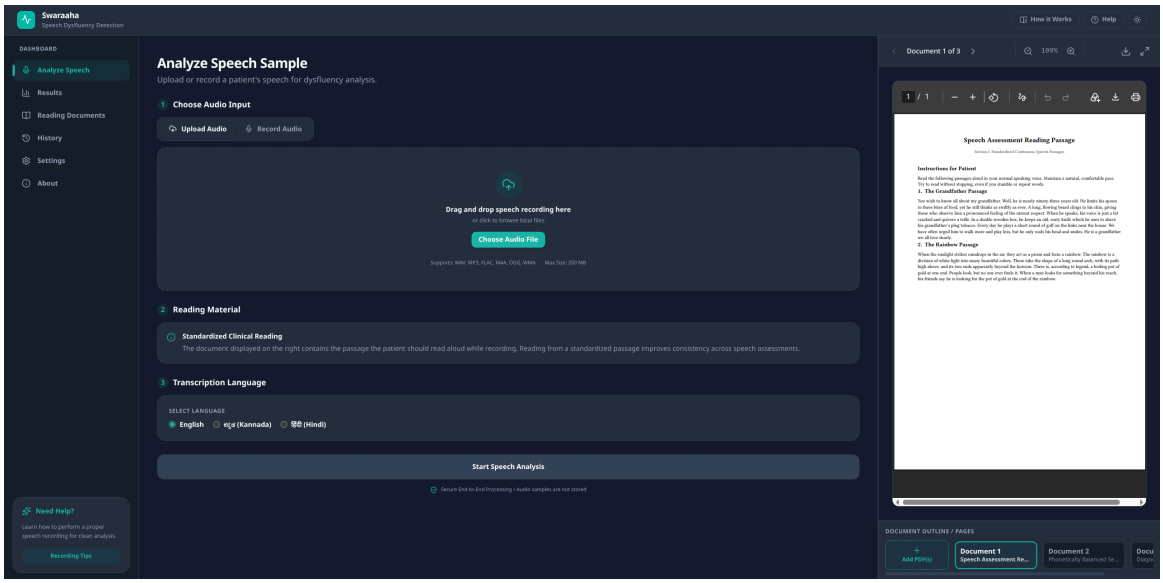


Figure 6.1: Home page of the web interface

The home page of the Swaraaha web application, shown in Figure 6.1, features the Analyze Speech Sample panel, which allows users to upload a supported audio file or record speech directly, select the transcription language, and begin the analysis. A reading-material viewer is provided alongside the input controls so that users can read a standardized passage before recording.

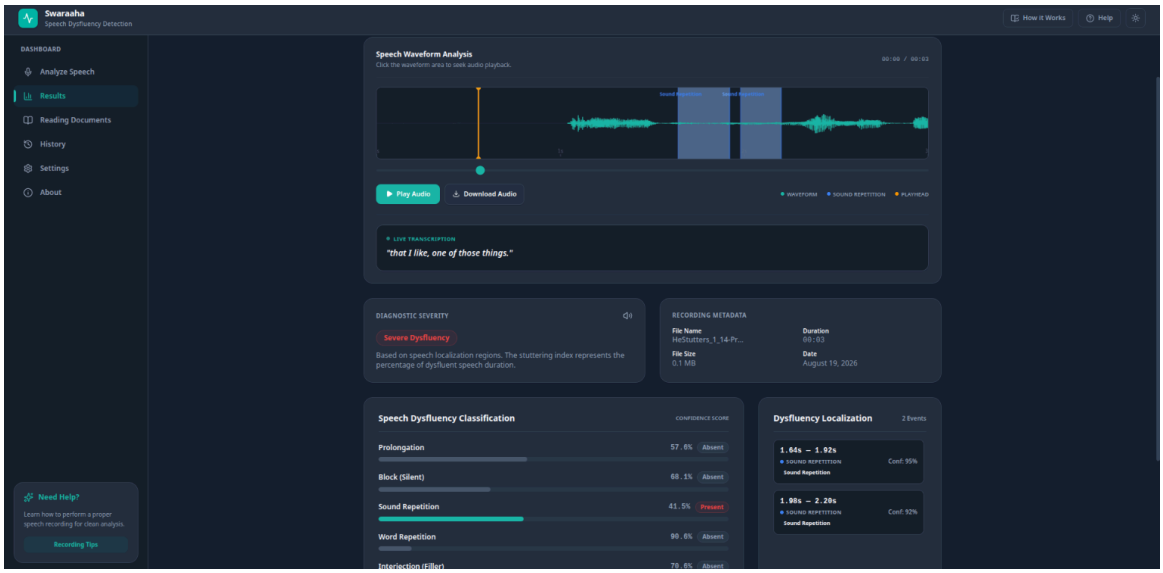


Figure 6.2: Results page of the web interface

The web results page, shown in Figure 6.2, presents the recorded waveform with localized dysfluency regions, synchronized playback controls, a timestamped live transcrip-

tion, and confidence scores for prolongation, block, sound repetition, word repetition, and interjection. Users can upload another sample, review the complete analysis, or download a PDF report.

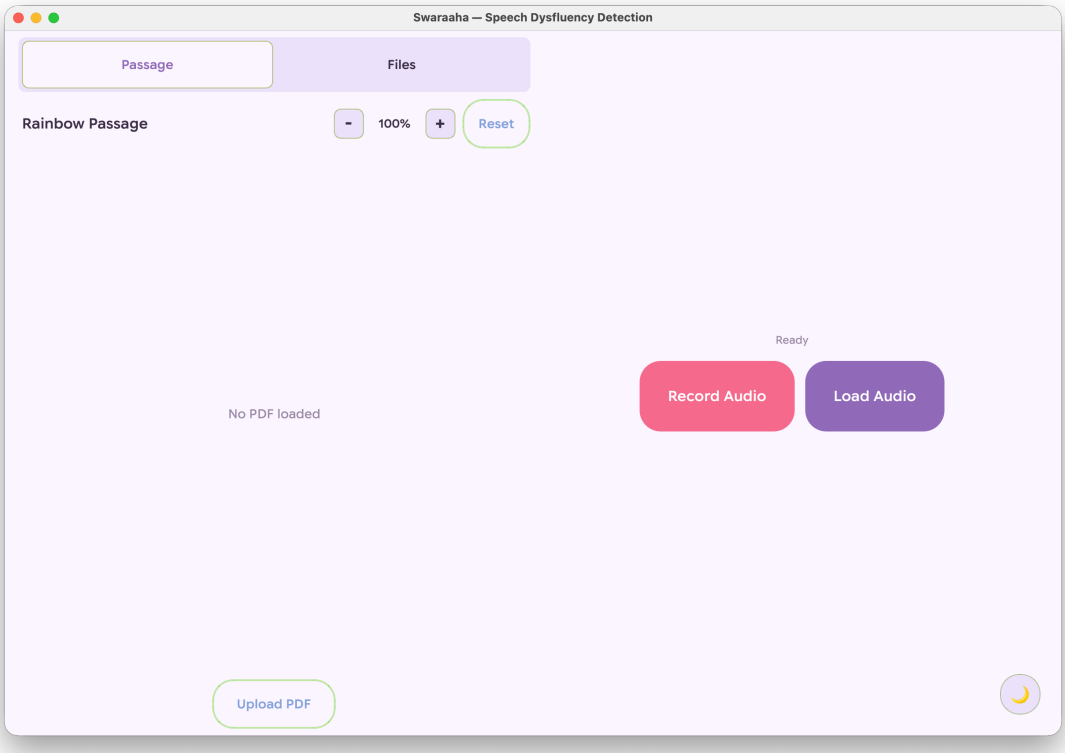


Figure 6.3: Home page of the desktop application

The home page of the desktop application, shown in Figure 6.3, allows users to open a standardized reading passage, load an audio file from the local file system, or record new speech. The interface also displays the current processing status and provides the controls required to continue once an audio sample has been selected. The desktop client mirrors the web workflow but runs entirely on the local machine, so it can be used in settings where a network connection is unreliable or where audio cannot leave the clinic. The reading passage viewer is embedded in the same window as the input controls, which means a speaker can read the standardized text and start recording without switching applications, reducing the variability introduced by different reading setups. The status indicator reports each processing stage in sequence, and the controls remain disabled until a valid audio sample is available, which prevents accidental submissions and makes the interaction predictable for non-technical users. Because the model package is imported directly rather than called over HTTP, the desktop application avoids the overhead of serializing audio and results, and it remains usable when the web backend is not running.

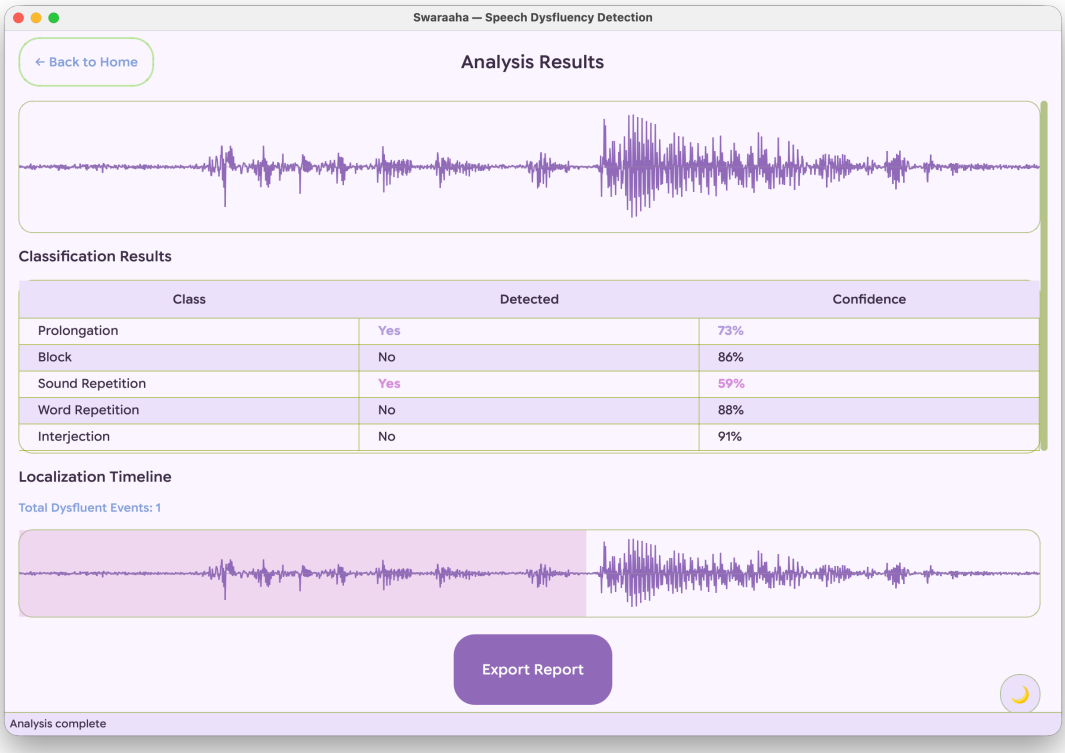


Figure 6.4: Analysis result page of the desktop application

The desktop analysis results page, shown in Figure 6.4, presents a classification table that reports whether each of the five dysfluency types was detected and the corresponding confidence value, while the localization timeline marks detected events on the waveform. The user can review the analysis, return to the home page, or export the results as a PDF report. The results page keeps the classification table and the timeline visually linked: selecting a row highlights the corresponding region on the waveform. This coupling lets a clinician move from a summary judgement to the exact evidence behind it without manually searching the audio, which is the main practical advantage of combining classification and localization in one view. The confidence values are shown per class rather than collapsed into a single score, so a weak detection is visible instead of being hidden. Exporting the PDF captures the table, waveform overlays, and transcript in one document. Together, the web and desktop screens present the same pipeline at different levels of deployment. The web interface emphasizes shared access and report download, while the desktop interface keeps audio and inference local. In both cases, the visible result connects the class-level prediction, the time-localized event, and the transcript context so that the output can be reviewed rather than treated as an unexplained label.

CHAPTER 7

CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS

7.1 CONCLUSION

The proposed stuttering classification and localization system supports accurate, objective assessment of speech dysfluencies through automated analysis of speech audio. Combining Wav2Vec 2.0 feature extraction with Wav2Vec2 frame-level localization and Whisper transcription strengthens feature extraction and improves classification performance. The pipeline covers data preprocessing, classification of five dysfluency types with five independent Wav2Vec 2.0 binary classifiers, event localization, transcription in English, Kannada, and Hindi, severity estimation, and PDF report generation.

The shipped default of independent classifiers reached a macro F1 score of 0.573, while the multitask shared-backbone variant scored 0.542 at the default threshold and 0.556 after per-class threshold tuning, with interjection the strongest class (F1 = 0.776, AUROC = 0.935) and block the weakest. The Wav2Vec2 localizer produced balanced frame-level results (precision 0.805, recall 0.738, F1 0.770) with a mean IoU of 0.779. Overall, the complete system achieves a detection accuracy of 0.586 for localizing dysfluency events, alongside a frame-level F1 of 0.770 and a macro F1 of up to 0.573 for classification.

The system gives clinicians, speech-language pathologists, and people who stutter consistent predictions that reduce the burden of manual analysis, limit diagnostic variability, and support timely assessment. Classification accuracy for sparse classes such as block and word repetition (macro F1 0.556) is not yet sufficient for a standalone diagnostic tool, and cross-corpus generalization weakens outside podcast-style data (multitask F1 drops from 0.556 on the test set to 0.238 on the Boli corpus). Despite these limitations, the modular architecture combines classification, localization, transcription, and reporting in one pipeline, providing a practical foundation for real-world speech assessment. The main contribution is therefore the integration of these stages into one usable workflow. A clinician can move from an uploaded recording to class-level probabilities, localized regions, aligned words, and a saved report without manually joining outputs from separate tools. The modular design also leaves room for later improvements to individual models without requiring the interface or reporting layer to be redesigned.

7.2 SCOPE FOR FUTURE ENHANCEMENTS

Looking ahead, future enhancements of the stuttering detection system could focus on expanding its functionality and improving detection accuracy. Incorporating larger and more diverse datasets from multiple clinical sources would help the model generalize better across different speakers, stuttering styles, and languages. The integration of additional deep learning architectures, such as larger pretrained models like Wav2Vec2-large and HuBERT-large, and advanced localization techniques such as curriculum learning, harder negative mining, and boundary-aware loss functions, could further refine the detection and localization of dysfluency events. Moreover, developing a user-friendly web and mobile interface for clinicians could make the system more accessible in real-time therapy settings. The system could also be enhanced with explainable AI features, such as attention visualization and saliency maps, to provide visual insights into how predictions are made, increasing trust and transparency in clinical decision-making. Multilingual models like MMS and XLS-R could extend language coverage beyond English, Kannada, and Hindi, while synthetic dysfluent speech generated with LLMs and text-to-speech systems could address the data scarcity problem during training. Finally, integrating this system with clinic databases and electronic health record systems could enable seamless clinical deployment, supporting doctors and speech-language pathologists in early assessment and personalized treatment planning for stuttering. Any such deployment should retain the current emphasis on privacy, transparent confidence values, and reviewable evidence. Future versions can improve coverage and accuracy while continuing to show how each result relates to the waveform, transcript, and detected dysfluency type.

REFERENCES

1. P. Arbajian et al., “Effect of Speech Segment Samples Selection in Stutter Block Detection and Remediation,” in Proc. of the International Conference on Speech and Computer, Springer, 2023, pp. 1-10.

2. V. Mitra et al., “Analysis and Tuning of a Voice Assistant System for Dysfluent Speech,” in Proc. Interspeech, 2021, pp. 3695-3699.

3. P. Mohapatra et al., “Speech Disfluency Detection with Contextual Representation and Data Distillation,” arXiv preprint arXiv:2306.05240, 2023.

4. J. Liu et al., “Automatic Speech Disfluency Detection Using Wav2Vec 2.0 for Different Languages with Variable Lengths,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2023, pp. 1-6.

5. A. Romana et al., “Automatic Disfluency Detection from Untranscribed Speech,” in Proc. Interspeech, 2024, pp. 1-5.

6. D. Wagner, S. P. Bayerl, I. Baumann, K. Riedhammer, E. Nöth, and T. Bocklet, “Large Language Models for Dysfluency Detection in Stuttered Speech,” in Proc. Interspeech, 2024, pp. 1-5.

7. S. A. Sheikh et al., “Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning,” IEEE Access, vol. 12, 2024.

8. X. Zhou et al., “YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection,” in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2024.

9. X. Zhou et al., “Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024, pp. 1039-1046, doi: 10.1109/SLT61566.2024.10832222.

10. J. Zhang et al., “Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection,” in Proc. Interspeech, 2024.

11. S. Kim and A. Kumar, “FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2022, pp. 784-790.

12. V. Uloza et al., “An Artificial Intelligence-Based Algorithm for the Assessment of Substitution Voicing,” Journal of Voice, vol. 38, no. 3, pp. 512-520, 2024.

13. H. Müller and C. Lee, “Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis,” Brain and Language, vol. 238, 2023.

14. A. Baevski et al., “Wav2Vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 12449-12460.

15. P. Khanna et al., “StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies,” arXiv preprint arXiv:2409.12345, 2024.

16. C. Lea and V. Mitra, “SEP-28K: A Dataset for Stuttering Event Detection from Podcasts,” in Proc. Interspeech, 2023.

17. O. Shonibare et al., “Enhancing ASR for Stuttered Speech with Limited Data Using Detect and Pass,” in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2024.

18. S. P. Bayerl, D. Wagner, E. Nöth, and K. Riedhammer, “Detecting Dysfluencies in Stuttering Therapy Using Wav2Vec 2.0,” in Proc. Interspeech, 2022, pp. 2878-2882.

19. S. Bayerl et al., “Dysfluencies Seldom Come Alone: Detection as a Multi-Label Problem,” in Proc. Int. Conf. Statistical Language and Speech Processing, 2023.

20. R. Gong et al., “AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024.

21. X. Liu et al., “An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024.

22. A. Batra et al., “Boli: A Dataset for Understanding Stuttering Experience,” arXiv preprint arXiv:2407.09876, 2024.

23. A. R. Valente et al., “Clinical Annotations for Automatic Stuttering Severity Assessment,” Journal of Communication Disorders, vol. 105, 2023.

24. T. Grósz et al., “Wav2Vec2-Based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering,” in Proc. Interspeech, 2023.

25. J. Tang et al., “Speech Annotation Guidelines with People Who Stutter,” Stammering Research, vol. 19, no. 1, 2024.

26. A. Romana et al., “FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition,” Language Resources and Evaluation, 2024.

27. L. Nie et al., “MMSD-Net: Towards Multi-Modal Stuttering Detection,” in Proc. ACM Multimedia, 2024.

28. R. P. Buzzeti et al., “Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese,” Speech Communication, vol. 158, 2024.

29. FastAPI, “FastAPI: Modern, Fast Web Framework for Building APIs with Python,” 2024. [Online]. Available: https://fastapi.tiangolo.com

30. Hugging Face, “Transformers: State-of-the-Art Machine Learning for PyTorch, TensorFlow, and JAX,” 2024. [Online]. Available: https://huggingface.co/docs/transformers

31. React, “React: A JavaScript Library for Building User Interfaces,” Meta, 2024. [Online]. Available: https://react.dev

PERSONAL PROFILE

<table>
<tr>
<td>  </td>
<td>

Prof. Ajay Shastry C G

- An educator with eight years of experience, his research and teaching interests center around Cybersecurity, AI/ML, and the Internet of Things.
- Completed B.E in Computer Science at Vivekananda College of Engineering & Technology, Puttur from 2012 - 2016 and M. Tech in Computer Science at Sahyadri College of Engineering and Management, Mangalore from 2016 - 2018.

Email ID: ajayshastrycg.ai@vcetputtur.ac.in

Phone: +91 99021 94905

</td>
</tr>
<tr>
<td>  </td>
<td>

K Shreekrishna Upadhyaya

USN: 4VP23AI020

Email ID: upadhyayashreekrishna@gmail.com

Phone: +91 96861 08613

</td>
</tr>
<tr>
<td>  </td>
<td>

M Chethan Keshav Bhat

USN: 4VP23AI023

Email ID: chethanbhat777@gmail.com

Phone: +91 80734 68867

</td>
</tr>
<tr>
<td>  </td>
<td>

Skanda Prasad K

USN: 4VP23AI051

Email ID: skandaprasad02@gmail.com

Phone: +91 87628 89622

</td>
</tr>
</table>



Srinivas Hegde M

USN: 4VP23AI054

Email ID: hegdesrinivasm@gmail.com

Phone: +91 90601 59605